

UNIVERSIDAD PRIVADA DEL NORTE

FACULTAD DE INGENIERÍA



CARRERA DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

CURSO: TECNICAS DE PROGRAMACIÓN ORIENTADA A OBJETOS

INFORME FINAL

“SISTEMAS DE CITAS MEDICAS PARA CLINICA”

INTEGRANTES:

- | | |
|--------------------------------|-----------|
| • QUISPE PUSCAN MARVIN HARLEY | N00429145 |
| • LEVANO DIAZ EVELYN GISELA | N00438718 |
| • CHOQUE MARTINEZ JOSÉ CARLOS | N00441596 |
| • MENDOZA YUCRA LIZ JACQUELINE | N00541462 |

DOCENTE: ING. MARTIN EDUARDO TORRES RODRIGUEZ

Lima, mayo del 2026

INDICE

UNIVERSIDAD PRIVADA DEL NORTE.....	1
FACULTAD DE INGENIERÍA	1
CAPÍTULO I: INTRODUCCIÓN	6
1.1 Marco Teórico	7
1.1.1 Programación Orientada a Objetos.....	7
1.1.1.1 Encapsulamiento.....	7
1.1.1.2 Herencia.....	7
1.1.1.3 Polimorfismo	8
1.1.1.4 Abstracción	8
1.1.2 Java como Lenguaje Orientado a Objetos	8
1.1.3 Principios SOLID y Código Limpio.....	9
1.1.4 Modelado con UML y Diagrama de Clases	9
1.1.5 Control de Versiones con Git y GitHub	9
1.1.6 Bases de Datos y JDBC	10
1.1.7 Interfaces Gráficas en Java: Swing y JavaFX	10
CAPÍTULO II: IDENTIFICACION DEL PROBLEMA.....	11
2.1. Problema general	11
2.2. Problemas específicos.....	11
2.3 Justificación	12
2.3.1 Justificación Practica	12
2.3.2 Justificación Académica	12
2.3.3 Justificación Tecnológica	12
CAPÍTULO III: ANÁLISIS DEL PROBLEMA	13
3.1 Descripción del Contexto	13
3.2 Análisis de Causa-Efecto (Diagrama de Ishikawa)	13
3.3 Impacto del Problema	14
CAPÍTULO IV: ANTECEDENTES.....	15
2.1 Antecedentes Internacionales	15
2.2 Antecedentes Nacionales	16
2.3 Antecedentes Relacionados con Seguridad de la Información.....	16
CAPÍTULO V: RESTRICCIONES DEL PROYECTO	18
5.1 Restricciones Técnicas	18
5.2 Restricciones Arquitectónicas	19
5.3 Restricciones de Seguridad.....	19

5.4 Restricciones Económicas	20
5.5 Restricciones Operativas	21
5.6 Restricciones de Escalabilidad	21
5.7 Restricciones de Tiempo	22
5.8 Restricciones de Recursos	22
5.9 Riesgos Tecnológicos	23
CAPÍTULO VI: ALTERNATIVAS DE SOLUCION	24
6.1 Introducción.....	24
6.2 Alternativa 1: Gestión Manual de Citas Médicas	24
6.3 Alternativa 2: Sistema Web de Gestión Médica.....	25
6.4 Alternativa 3: Sistema de Escritorio en Java Swing.....	26
6.5 Cuadro Comparativo de Alternativas	27
6.6 Alternativa Seleccionada	27
6.7 Justificación Técnica de la Solución Elegida	27
CAPÍTULO VII: OBJETIVOS	29
7.1 Objetivo General.....	29
7.2 Objetivos Específicos	29
CAPÍTULO VIII: ALCANCES DEL SISTEMA	30
8.1 Alcance General del Sistema.....	30
8.2 Alcance Funcional	30
8.2.1 Módulo de Gestión de Pacientes.....	30
8.2.2 Módulo de Gestión de Médicos	30
8.2.3 Módulo de Gestión de Especialidades	30
8.2.4 Módulo de Gestión de Horarios	30
8.2.5 Módulo de Gestión de Citas Médicas	31
8.2.6 Módulo de Gestión de Pagos	31
8.2.7 Módulo de Historial Clínico	31
8.2.8 Módulo de Reportes.....	31
8.2.9 Módulo de Seguridad y Acceso	31
8.3 Alcance Técnico	31
8.4 Usuarios del Sistema	31
8.5 Alcance Operativo	32
CAPÍTULO IX: REQUERIMIENTOS FUNCIONALES	32
9.1 Requerimientos Funcionales.....	32
9.2 Requerimientos No Funcionales.....	34

CAPÍTULO X. HISTORIAS DE USUARIO	35
CAPÍTULO XI. DISEÑO DEL SISTEMA	39
11.1 DIAGRAMA DE CLASES	39
11.1.1 Clase abstracta Persona.....	40
11.1.2 Herencia: Paciente y Medico extienden Persona	40
11.1.3 Clase Especialidad: asociación con Medico	41
11.1.4 Clase Cita: asociación central del sistema	41
11.1.5 Clase Urgencia: atención sin cita previa	41
11.1.6 Clases HistorialClinico y Pago: trazabilidad	41
11.1.7 Clase Horario: gestión de disponibilidad	41
11.1.8. Clase Usuario — control de acceso	41
11.2 Relaciones identificadas:	42
11.3 Patrón de diseño aplicado	42
CAPÍTULO XII: IMPLEMENTACIÓN	43
12.1 Lenguaje utilizado	43
12.2 Descripción del Sistema	43
12.2.1 Flujo operativo del sistema	45
12.3 Módulos Implementados	46
12.3.1 Módulo Acceso y Seguridad	46
12.3.2 Módulo de Gestión de Pacientes	47
12.3.3 Módulo de Gestión de Médicos	48
12.3.4 Módulo de Gestión de especialidades	49
12.3.5 Módulo de Gestión de Horarios	49
12.3.6 Módulo de Gestión de Citas Médicas	50
12.3.7 Módulo de Urgencias Médicas	51
12.3.8 Módulo de Historial Clínico	52
12.3.9 Módulo de Gestión de Pagos	52
12.3.10 Módulo de Reportes	53
12.3.11 Módulo de Validación y Persistencia	54
CAPÍTULO XIII: EVIDENCIA DEL SOFTWARE	55
13.1 Link de Github.....	55
13.2 Estructura General del proyecto	56
13.3 Implementación de paquetes.....	57
13.3.1 Paquete modelo	57
13.3.1.1 Persona.java – Abstracción y base de herencia	58

13.3.1.2 Paciente.java – Herencia y polimorfismo	60
13.3.1.3 Medico.java – Herencia y agregación.....	61
13.3.1.4 Usuario.java – Encapsulamiento y control de roles.....	61
13.3.1.5 Cita.java – Asociación y ciclo de vida.....	62
13.3.1.6 Urgencia.java – Atención sin cita previa	63
13.3.1.7 HistorialClinico.java – Trazabilidad clínica	64
13.3.1.8 Pago.java – Registro de transacciones	65
13.3.1.9 Especialidad.java – Soporte a selección en formularios.....	65
13.3.1.10 Horario.java – Disponibilidad del médico	66
13.3.2 Paquete datos	66
13.3.3 Paquete controlador	68
13.3.4 Paquete vista	70
13.3.5 Paquete Util.....	72
13.3.6 Paquete Principal	74
13.3.7 Interfaz general del proyecto modo Administrador	75
13.4 Capturas de ejecución del sistema y validación de historias de usuario implementados:	76
CAPÍTULO XIV: CONCLUSIONES	89
CAPÍTULO XV: REFERENCIAS BIBLIOGRÁFICAS	91

CAPÍTULO I: INTRODUCCIÓN

En el ámbito de los establecimientos de salud de pequeña escala, la gestión de citas médicas constituye un proceso crítico que determina la eficiencia operativa y la calidad del servicio brindado al paciente. Sin embargo, en la gran mayoría de clínicas y consultorios menores, dicha gestión se lleva a cabo mediante mecanismos manuales: cuadernos de registro, llamadas telefónicas no sistematizadas y hojas de cálculo sin integración entre sí. Esta realidad genera problemas recurrentes como la superposición de horarios, la pérdida de datos del paciente, la demora en la atención y la imposibilidad de obtener reportes consolidados que orienten la toma de decisiones administrativas.

Desde la perspectiva de la ingeniería de software, la Programación Orientada a Objetos (POO) constituye el paradigma más adecuado para modelar sistemas que reflejan entidades del mundo real. Este enfoque permite representar entidades como pacientes, médicos, citas y pagos como objetos con atributos y comportamientos definidos, favoreciendo la modularidad, la reutilización del código y la mantenibilidad del sistema (Deitel & Deitel, 2016). Tal como señala Joyanes Aguilar y Zahonero Martínez (2011), la POO facilita la abstracción de la realidad y proporciona una base sólida para el desarrollo de aplicaciones robustas y escalables.

El lenguaje Java se posiciona como una tecnología apropiada para este proyecto debido a su portabilidad multiplataforma, su robustez técnica, su amplio ecosistema de bibliotecas y su capacidad de integrar interfaces gráficas y bases de datos relacionales. Como afirma Sánchez González (2019), Java trabaja a base de objetos que representan la información, lo que lo convierte en una herramienta natural para implementar sistemas orientados a objetos de manera coherente y estructurada.

El presente informe documenta el análisis, diseño e implementación del Sistema de Citas Médicas para una Clínica Pequeña, desarrollado en el marco del curso de Técnicas de Programación Orientada a Objetos de la Universidad Privada del Norte. Este sistema integra los contenidos fundamentales del curso: clases, herencia, polimorfismo, encapsulamiento, colecciones, manejo de excepciones, interfaz gráfica, acceso a base de datos y control de versiones con Git y GitHub.

1.1 Marco Teórico

1.1.1 Programación Orientada a Objetos

La Programación Orientada a Objetos (POO) es un paradigma de programación que organiza el software en torno a entidades denominadas objetos, los cuales encapsulan tanto datos (atributos) como comportamientos (métodos). Este paradigma surge como respuesta a la creciente complejidad de los sistemas de software y a la necesidad de estructurar el código de manera modular, reutilizable y mantenible.

Según Joyanes Aguilar y Zahonero Martínez (2011), la POO *"favorece la reutilización del código, la abstracción de la realidad y la organización modular del software"*, lo que la convierte en el enfoque más adecuado para desarrollar sistemas complejos que reflejen entidades del mundo real. En el contexto del presente proyecto, la POO permite modelar directamente las entidades clínicas del sistema: pacientes, médicos, especialidades, citas, pagos e historiales.

1.1.1.1 Encapsulamiento

El encapsulamiento es el principio fundamental de la POO que consiste en ocultar los detalles internos de implementación de una clase y exponer únicamente la interfaz pública necesaria para interactuar con ella. Mediante los modificadores de acceso `private`, `protected` y `public`, Java controla el acceso a los atributos y métodos de cada clase. Sánchez González (2019) lo define como el mecanismo que consiste en ocultar los detalles internos de una clase y exponer o dar a conocer solo los detalles que sean necesarios para el resto de las clases, permitiendo restringir y controlar su uso. En el sistema propuesto, atributos como el DNI del paciente, el código del médico o el monto del pago serán declarados como privados y accedidos mediante métodos `getter` y `setter`.

1.1.1.2 Herencia

La herencia es el mecanismo mediante el cual una clase (subclase o clase derivada) puede adquirir los atributos y métodos de otra clase (superclase o clase base), extendiendo o especializando su comportamiento. Deitel y Deitel (2016) señalan que la herencia es una relación entre una clase más general y una más específica o especializada, que adquiere los comportamientos y atributos de la clase base. En el sistema de citas médicas, esta relación se refleja en la jerarquía de clases donde `Paciente` y `Medico` extienden de la clase base `Persona`,

heredando atributos comunes como nombre, apellido, DNI, teléfono y correo electrónico. Esta estructura minimiza la duplicación de código y favorece la coherencia del modelo de datos.

1.1.1.3 Polimorfismo

El polimorfismo es la capacidad de un objeto de asumir múltiples formas o comportamientos según el contexto en que es invocado. En Java, el polimorfismo se implementa principalmente mediante la sobrescritura de métodos (@Override) y la sobrecarga de métodos. Serbat Ocaña define el polimorfismo como una propiedad fundamental de la POO que permite que un mismo método se comporte de manera diferente dependiendo del tipo de objeto que lo ejecuta. En el sistema propuesto, el método mostrarDatos() es sobrescrito en las clases Paciente, Medico y Cita para retornar la información específica de cada entidad, manteniendo una interfaz común.

1.1.1.4 Abstracción

La abstracción permite modelar entidades complejas del mundo real simplificando su representación al considerar únicamente los atributos y comportamientos relevantes para el sistema. Deitel y Deitel (2016) describen las clases abstractas como representaciones que no pueden ser instanciadas directamente y que sirven como plantilla base para clases más específicas. En el contexto del sistema de citas médicas, la clase Persona constituye una abstracción que define los atributos comunes de todas las personas que interactúan con el sistema, sin ser instanciada directamente.

1.1.2 Java como Lenguaje Orientado a Objetos

Java es un lenguaje de programación orientado a objetos creado por Sun Microsystems en 1995, actualmente mantenido por Oracle. Su principal característica es la portabilidad multiplataforma, garantizada por la Máquina Virtual Java (JVM), que permite ejecutar el mismo programa compilado en cualquier sistema operativo sin modificaciones. Sánchez González (2019) destaca que Java trabaja a base de objetos que representan la información, creando instancias de una clase definida que cuentan con propiedades, atributos y métodos que los distinguen del resto, tal como los objetos en el mundo real.

En el contexto del desarrollo de sistemas empresariales, Java incorpora un ecosistema robusto de bibliotecas y frameworks que facilitan la implementación de interfaces gráficas (Swing, JavaFX), la conexión a bases de datos (JDBC), el manejo de colecciones (ArrayList, HashMap) y el control de excepciones. Estas capacidades lo convierten en una plataforma integral para el desarrollo del sistema de citas médicas propuesto.

1.1.3 Principios SOLID y Código Limpio

Los principios SOLID constituyen un conjunto de cinco directrices de diseño de software orientado a objetos cuyo objetivo es mejorar la mantenibilidad, extensibilidad y testabilidad del código. Según Gómez (2020), el acrónimo SOLID comprende: Responsabilidad Única (SRP), Abierto/Cerrado (OCP), Sustitución de Liskov (LSP), Segregación de Interfaces (ISP) e Inversión de Dependencias (DIP). La aplicación de estos principios permite desarrollar un sistema clínico robusto donde cada clase tiene una responsabilidad claramente definida y el sistema puede extenderse sin modificar código existente.

El concepto de código limpio, popularizado por Robert C. Martin, establece que el código debe estar escrito con la intención de que otra persona lo entienda fácilmente. Gómez (2020) cita a Carlos Blé al definir código limpio como aquel que se ha escrito con la intención de que otra persona (o tú mismo en el futuro) lo entienda. En el sistema propuesto, esto se traduce en nombres de variables y métodos descriptivos, clases con responsabilidades únicas y métodos de tamaño reducido.

1.1.4 Modelado con UML y Diagrama de Clases

El Lenguaje de Modelado Unificado (UML) es un estándar internacional para la representación visual de sistemas de software. El diagrama de clases UML es la herramienta central para modelar la estructura estática de un sistema orientado a objetos, representando las clases, sus atributos, métodos y las relaciones entre ellas. Deitel y Deitel (2016) señalan que en UML cada clase es representada en tres secciones: nombre, atributos y métodos, y que la comunicación entre clases se establece a través de relaciones.

Las relaciones fundamentales entre clases incluyen la asociación (relación general), la herencia o generalización (relación es-un), la agregación (relación tiene-un con partes independientes) y la composición (relación tiene-un con partes dependientes). En el sistema de citas médicas, estas relaciones se manifiestan en que Paciente y Medico heredan de Persona, una Cita está compuesta por un Paciente y un Médico, y un Historial depende de una Cita.

1.1.5 Control de Versiones con Git y GitHub

Git es un sistema de control de versiones distribuido que permite registrar el historial de cambios del código fuente, gestionar ramas de desarrollo paralelas y facilitar la colaboración entre equipos. Durán García (2023) define el control de versiones como un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo, de modo que

sea posible recuperar versiones específicas más adelante. Para el desarrollo colaborativo del sistema de citas médicas, Git garantiza la trazabilidad del trabajo de cada integrante del equipo.

GitHub complementa estas capacidades al proporcionar un repositorio remoto donde el equipo puede centralizar el código, realizar revisiones mediante pull requests y mantener evidencias del avance del proyecto. Moure (2024) destaca que GitHub promueve buenas prácticas de trabajo colaborativo al facilitar la revisión de código, la gestión de issues y la documentación del proyecto mediante archivos README. El uso de Git Flow como estrategia de ramificación permite organizar el desarrollo en ramas separadas para cada funcionalidad del sistema.

1.1.6 Bases de Datos y JDBC

Java Database Connectivity (JDBC) es una API estándar de Java que permite a las aplicaciones interactuar con bases de datos relacionales de manera uniforme, independientemente del sistema de gestión de base de datos utilizado. Mediante JDBC, el sistema de citas médicas puede realizar operaciones CRUD (Create, Read, Update, Delete) sobre las tablas que representan pacientes, médicos, citas, horarios y pagos. La conexión se establece mediante un driver específico del motor de base de datos, en este caso MySQL o SQLite, lo que garantiza la portabilidad de la solución.

1.1.7 Interfaces Gráficas en Java: Swing y JavaFX

Java ofrece dos bibliotecas principales para el desarrollo de interfaces gráficas de escritorio: Swing y JavaFX. Swing es la biblioteca tradicional de Java para interfaces gráficas, mientras que JavaFX es la alternativa moderna que ofrece mayor riqueza visual y soporte para multimedia. Joyanes Aguilar y Zahonero Martínez (2011) señalan que las interfaces gráficas permiten al usuario interactuar con el sistema de manera intuitiva mediante componentes como ventanas, botones, campos de texto, tablas y menús. En el sistema de citas médicas, la interfaz gráfica constituye el punto de interacción principal entre los usuarios (recepcionistas, médicos, administradores y cajeros) y la lógica del negocio.

CAPÍTULO II: IDENTIFICACION DEL PROBLEMA

El sistema actual de gestión de citas médicas en clínicas pequeñas presenta múltiples deficiencias operativas derivadas del uso de procesos manuales y herramientas no integradas. En muchos establecimientos de salud, el registro de pacientes, médicos y horarios se realiza mediante cuadernos físicos, llamadas telefónicas o archivos de cálculo independientes, lo que genera desorganización y pérdida de información.

La ausencia de una plataforma centralizada ocasiona problemas frecuentes como superposición de citas, errores en la programación de horarios, dificultades para consultar historiales de atención y falta de control sobre los pagos realizados por los pacientes. Estas deficiencias afectan directamente la calidad del servicio médico y la eficiencia administrativa de la clínica.

Asimismo, la inexistencia de reportes automatizados limita la capacidad de la administración para realizar seguimiento de ingresos, controlar la productividad médica y tomar decisiones oportunas basadas en información confiable.

Desde la perspectiva tecnológica y académica, resulta necesario desarrollar un sistema informático basado en Programación Orientada a Objetos utilizando Java, permitiendo automatizar los procesos clínicos esenciales, mejorar la trazabilidad de la información y fortalecer la gestión operativa de la clínica.

2.1. Problema general

¿Cómo desarrollar un sistema de citas médicas para una clínica pequeña, implementado en Java bajo el paradigma de la Programación Orientada a Objetos, que permita registrar, organizar y controlar pacientes, médicos, horarios, citas y pagos de manera eficiente, centralizada y confiable?

2.2. Problemas específicos

- La programación manual de citas genera cruces de horario que afectan la atención al paciente y la reputación del establecimiento de salud.
- La información de pacientes y médicos no se encuentra centralizada, lo que dificulta la consulta y actualización de datos en tiempo real.
- La consulta de historiales básicos de atención resulta lenta, desordenada e incompatible con una gestión clínica eficiente.

- No existe un control estructurado de pagos asociados a las atenciones, lo que impide el seguimiento de ingresos y genera inconsistencias financieras.
- La ausencia de reportes automatizados dificulta la gestión administrativa y la toma de decisiones oportunas por parte de la dirección de la clínica.

2.3 Justificación

2.3.1 Justificación Practica

El desarrollo de un sistema de citas médicas responde a una necesidad real y frecuente en establecimientos de salud de pequeña escala en el contexto peruano. La implementación de una solución informática permite reemplazar los procesos manuales por flujos de trabajo automatizados, lo que reduce significativamente los errores administrativos, optimiza la disponibilidad de recursos médicos y mejora la experiencia del paciente. Un sistema centralizado de información asegura que los datos sean consistentes, accesibles y seguros.

2.3.2 Justificación Académica

Desde el punto de vista académico, este proyecto representa la integración aplicada de los contenidos fundamentales del curso. La solución propuesta pone en práctica conceptos como la definición de clases y objetos, el encapsulamiento de atributos, la herencia entre clases base y derivadas, el polimorfismo en métodos, el manejo de colecciones con ArrayList, el control de excepciones, la implementación de interfaces gráficas y el acceso a bases de datos mediante JDBC (Joyanes Aguilar & Zahonero Martínez, 2011). Todos estos elementos constituyen competencias centrales del perfil del ingeniero de sistemas.

2.3.3 Justificación Tecnológica

La elección de Java como plataforma de desarrollo se sustenta en su sólida orientación a objetos, su madurez tecnológica y su amplia aceptación en el sector empresarial e institucional. Tal como señala Sánchez González (2019), Java permite crear instancias de clases con propiedades, atributos y métodos que los distinguen del resto, tal como los objetos en el mundo real. Esta capacidad hace de Java el lenguaje idóneo para modelar entidades clínicas como pacientes, médicos y citas. Adicionalmente, la aplicación de principios de código limpio y buenas prácticas de desarrollo incluyendo los principios SOLID garantiza que el sistema sea mantenible, legible y escalable (Gómez, 2020).

CAPÍTULO III: ANÁLISIS DEL PROBLEMA

3.1 Descripción del Contexto

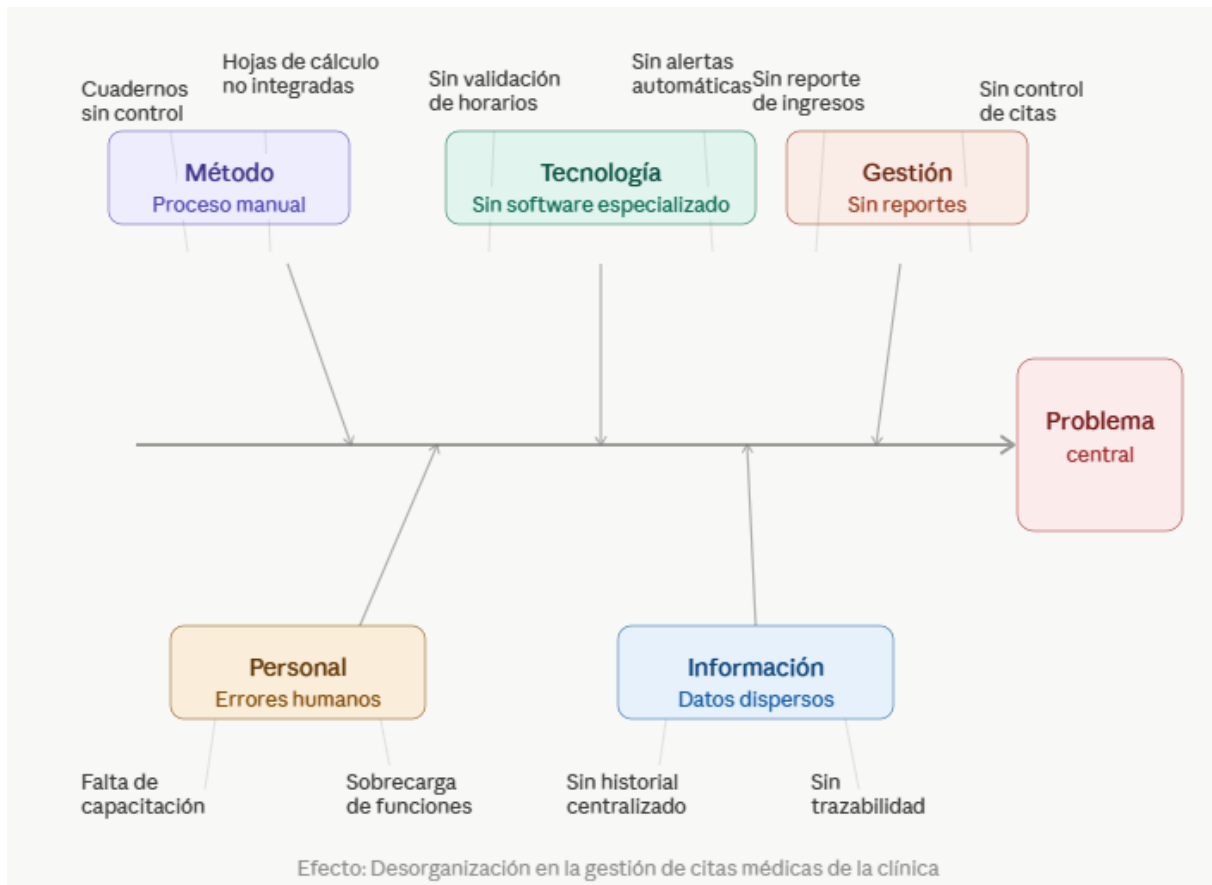
Las clínicas pequeñas en el contexto peruano operan generalmente con recursos humanos y tecnológicos limitados. La gestión administrativa recae en un número reducido de personas — en la mayoría de casos, recepcionistas que atienden simultáneamente llamadas telefónicas, registran citas, orientan a los pacientes y coordinan con los médicos. Esta sobrecarga operativa, combinada con la ausencia de herramientas tecnológicas adecuadas, genera un entorno propenso a errores e ineficiencias.

3.2 Análisis de Causa-Efecto (Diagrama de Ishikawa)

El análisis de causas del problema central la desorganización en la gestión de citas médicas se estructura a través del diagrama de causa y efecto, también conocido como diagrama de Ishikawa. Este modelo identifica las causas raíz desde cinco dimensiones:

- **Método:** Uso de cuadernos de registro, llamadas telefónicas sin sistematización y hojas de cálculo no integradas. La ausencia de un flujo de trabajo estandarizado genera inconsistencias en el registro de información.
- **Personal:** Errores humanos al registrar datos, horarios y pacientes. La falta de capacitación y la sobrecarga de funciones aumentan la probabilidad de equivocaciones.
- **Tecnología:** Ausencia de software especializado para la gestión clínica. No se dispone de herramientas que validen horarios, detecten cruces de citas o automaticen notificaciones.
- **Información:** Datos dispersos en múltiples fuentes no integradas. La falta de centralización impide obtener una visión consolidada del estado de la clínica.
- **Gestión:** Inexistencia de reportes periódicos de citas, ingresos y rendimiento médico, lo que dificulta la toma de decisiones administrativas oportunas.

Imagen del Diagrama de Ishikawa



3.3 Impacto del Problema

Los problemas identificados generan un impacto negativo tanto en la calidad del servicio al paciente como en la eficiencia operativa de la clínica. La superposición de citas, la pérdida de información y la imposibilidad de consultar historiales básicos resultan en tiempos de espera prolongados, insatisfacción del paciente y potenciales riesgos para la atención médica oportuna. Desde la perspectiva financiera, la ausencia de control de pagos genera inconsistencias en los ingresos y dificulta el seguimiento de la rentabilidad de la clínica.

CAPÍTULO IV: ANTECEDENTES

Diversos estudios han demostrado que la implementación de sistemas informáticos en establecimientos de salud mejora significativamente la organización administrativa y la calidad de atención al paciente. Según Deitel y Deitel (2016), los sistemas orientados a objetos permiten modelar entidades reales de manera estructurada, facilitando la automatización de procesos complejos.

Joyanes Aguilar y Zahonero Martínez (2011) sostienen que la Programación Orientada a Objetos favorece la reutilización del código, la mantenibilidad y la escalabilidad de los sistemas empresariales, siendo especialmente adecuada para aplicaciones clínicas donde existen múltiples relaciones entre entidades como pacientes, médicos y citas.

Por otro lado, Gómez (2020) señala que la aplicación de principios SOLID y código limpio permite desarrollar sistemas más robustos y fáciles de mantener, reduciendo costos de mantenimiento y aumentando la calidad del software.

En el contexto de la digitalización de servicios médicos, múltiples clínicas pequeñas han migrado de procesos manuales hacia sistemas automatizados, logrando disminuir errores administrativos, mejorar la trazabilidad de la información y optimizar la gestión de horarios y pagos.

2.1 Antecedentes Internacionales

Veeraiyah et al. (2026) desarrollaron un sistema inteligente de gestión hospitalaria basado en tecnología web denominado *Smart Web Based Hospital Management System*, cuyo objetivo fue optimizar la administración hospitalaria mediante la automatización de procesos como registro de pacientes, programación de citas médicas, asignación de camas y control de recursos hospitalarios. El estudio utilizó tecnologías Java Spring Boot, MySQL y herramientas web como HTML, CSS y JavaScript. Entre los principales resultados obtenidos se identificó una reducción significativa de procesos manuales, mejora en la disponibilidad de información en tiempo real y optimización de la experiencia del paciente. Además, el sistema incorporó un módulo de emergencias capaz de analizar la disponibilidad de recursos médicos en tiempo real, constituyendo una propuesta escalable para hospitales de mayor complejidad. Los autores concluyen que la automatización hospitalaria mejora la eficiencia operativa, disminuye errores administrativos y fortalece la capacidad de respuesta médica.

Sánchez (2023) desarrolló un sistema de gestión de historias clínicas odontológicas y periodontales en Ecuador, con el propósito de mejorar la administración de información clínica y apoyar investigaciones epidemiológicas dentro de la UCACUE. La investigación aplicó metodología SCRUM y un enfoque de desarrollo ágil orientado a la satisfacción del usuario. Los resultados demostraron mejoras en la eficiencia operativa, facilidad de uso, rapidez de respuesta y optimización de procesos clínicos. Asimismo, el estudio identificó la necesidad de fortalecer la seguridad de la información y ampliar funcionalidades en futuras versiones del sistema. El autor concluye que la implementación de soluciones tecnológicas contribuye significativamente a la modernización de los servicios médicos y a la mejora de la toma de decisiones institucionales

2.2 Antecedentes Nacionales

Mogollón Moreno (2025) realizó una propuesta de implementación de un sistema de gestión de citas médicas para la Clínica Virgen de la Puerta en Tumbes, con el objetivo de optimizar el registro de información y mejorar la atención a los pacientes. La investigación utilizó un enfoque cuantitativo, diseño no experimental y metodología SCRUM para el desarrollo de la propuesta tecnológica. Los resultados evidenciaron que el 70 % de los pacientes manifestaba insatisfacción con el sistema actual debido a retrasos, duplicidad de registros y desorganización administrativa. Asimismo, el 80 % consideró favorable la implementación de un sistema digital para mejorar la programación de citas médicas. El estudio concluye que la automatización de procesos médicos constituye una alternativa viable para modernizar la gestión hospitalaria y elevar la calidad del servicio.

Ramírez y Vásquez (2022) implementaron un sistema digital de gestión de citas médicas en el Hospital Nacional Arzobispo Loayza con la finalidad de mejorar la eficiencia en la atención de pacientes. La investigación tuvo un enfoque cuantitativo con diseño experimental y una muestra de 500 pacientes. Los resultados mostraron una reducción del 35 % en los tiempos de espera y un incremento del 92 % en el nivel de satisfacción de los usuarios. Los autores concluyen que los sistemas digitales permiten optimizar recursos hospitalarios, mejorar la organización institucional y elevar la calidad de atención médica brindada al paciente.

2.3 Antecedentes Relacionados con Seguridad de la Información

Llanos Cardona, López y Mejía-Lobo (2023) desarrollaron un estudio sobre la implementación de un Sistema de Gestión de Seguridad de la Información en una entidad del sector salud utilizando la norma ISO/IEC 27001 y el modelo MSPI. La investigación tuvo como finalidad

garantizar la confidencialidad, integridad y disponibilidad de los activos de información hospitalarios mediante un enfoque basado en gestión de riesgos y mejora continua. Los resultados evidenciaron que la institución evaluada presentaba un bajo nivel de implementación de controles de seguridad, obteniendo una calificación promedio de 22 sobre 100 en el diagnóstico de seguridad de la información. Los autores concluyen que la implementación de políticas de seguridad, controles técnicos y procesos basados en ISO 27001 permite proteger adecuadamente la información clínica y reducir riesgos asociados a vulnerabilidades digitales dentro de instituciones de salud.

Asimismo, el estudio destaca la importancia de proteger la historia clínica digital mediante mecanismos de seguridad informática, control de acceso y políticas de protección de datos personales, debido a que la información médica constituye un activo crítico dentro de las instituciones hospitalarias.

CAPÍTULO V: RESTRICCIONES DEL PROYECTO

5.1 Restricciones Técnicas

El desarrollo del Sistema de Citas Médicas presenta diversas restricciones técnicas derivadas del alcance académico, las tecnologías seleccionadas y las limitaciones propias del entorno de implementación.

Restricciones Técnicas Generales

- El sistema ha sido desarrollado exclusivamente utilizando el lenguaje de programación Java aplicando los principios de Programación Orientada a Objetos (POO), priorizando encapsulamiento, herencia, modularidad y reutilización de código.
- La interfaz gráfica del sistema fue implementada mediante Java Swing, por lo que la solución está orientada a aplicaciones de escritorio y no a arquitecturas web o móviles.
- La persistencia de datos se realiza mediante almacenamiento local utilizando archivos estructurados y componentes de acceso a datos desarrollados específicamente para el proyecto.
- El sistema depende del entorno de ejecución Java (JRE), por lo que requiere que el equipo cliente tenga instalada una versión compatible de Java.
- Algunas funcionalidades visuales, como el manejo de fechas, utilizan librerías externas como JCalendar, lo que genera dependencia tecnológica adicional.

Explicación Técnica

La elección de Java Swing responde a la necesidad de desarrollar una solución estable y orientada al aprendizaje de estructuras orientadas a objetos dentro del curso de Técnicas de Programación Orientada a Objetos. Asimismo, la arquitectura local reduce la complejidad asociada a redes, despliegues distribuidos y servicios web, permitiendo concentrar el desarrollo en la lógica del negocio y la organización modular del sistema.

5.2 Restricciones Arquitectónicas

La arquitectura del sistema ha sido diseñada bajo un enfoque modular compuesto por paquetes organizados en modelo, vista, controlador y datos. Sin embargo, debido al alcance académico del proyecto, existen limitaciones estructurales importantes.

Restricciones Arquitectónicas Identificadas

- El sistema no implementa arquitecturas distribuidas ni servicios basados en microservicios.
- La solución no utiliza frameworks empresariales avanzados como Spring Boot, Hibernate o Jakarta EE.
- La comunicación entre módulos se realiza de manera local dentro de la misma aplicación de escritorio.
- No se implementa separación física entre cliente y servidor.
- El sistema no incorpora balanceo de carga ni mecanismos avanzados de concurrencia multiusuario.
- La arquitectura actual está optimizada para clínicas pequeñas y ambientes locales de operación.

Explicación Arquitectónica

La arquitectura modular implementada busca facilitar el mantenimiento y organización del código fuente, permitiendo separar claramente la lógica de negocio, la interfaz gráfica y la persistencia de datos. No obstante, el sistema no está orientado a escenarios empresariales de gran escala debido a las restricciones de tiempo y complejidad propias del proyecto académico.

5.3 Restricciones de Seguridad

El sistema incorpora mecanismos básicos de autenticación y validación de datos; sin embargo, no implementa estándares avanzados de ciberseguridad utilizados en entornos hospitalarios empresariales.

Restricciones de Seguridad Identificadas

- La autenticación de usuarios es básica y no incorpora cifrado avanzado de contraseñas.
- No se implementan mecanismos de autenticación multifactor (MFA).

- El sistema no utiliza protocolos de seguridad empresarial como OAuth 2.0, JWT o LDAP.
- No se incluye auditoría avanzada de eventos ni trazabilidad completa de operaciones críticas.
- La información almacenada localmente podría verse afectada ante accesos no autorizados al equipo físico.
- No se implementan mecanismos automáticos de respaldo ni recuperación ante desastres.

Explicación de Seguridad

Las restricciones de seguridad responden principalmente al alcance académico del proyecto y a la prioridad otorgada al aprendizaje de programación orientada a objetos y diseño modular. La incorporación de mecanismos avanzados de ciberseguridad requeriría infraestructura especializada y mayor complejidad arquitectónica.

5.4 Restricciones Económicas

El proyecto ha sido desarrollado considerando limitaciones presupuestarias propias de un entorno académico universitario.

Restricciones Económicas Identificadas

- El desarrollo utiliza herramientas gratuitas o de licencia académica como Eclipse IDE, GitHub y PlantUML.
- No se contempla adquisición de servidores empresariales ni infraestructura dedicada.
- No se considera contratación de servicios cloud comerciales.
- El proyecto no dispone de financiamiento externo ni patrocinio empresarial.
- Las pruebas y ejecución del sistema se realizan utilizando equipos personales de los integrantes del proyecto.

Explicación Económica

La utilización de tecnologías gratuitas permite reducir costos y garantizar accesibilidad al entorno de desarrollo, favoreciendo el aprendizaje y la implementación práctica sin depender de licencias empresariales de alto costo.

5.5 Restricciones Operativas

El sistema ha sido diseñado específicamente para clínicas pequeñas o consultorios privados con operaciones administrativas básicas.

Restricciones Operativas Identificadas

- El sistema está orientado exclusivamente a clínicas pequeñas y no a hospitales de gran escala.
- No se contempla integración con plataformas gubernamentales como SUNAT o RENIEC.
- No se incluye interoperabilidad con sistemas hospitalarios externos.
- El sistema no incorpora funcionalidades de telemedicina ni videoconferencias médicas.
- No se implementa gestión farmacéutica, inventario médico ni control de laboratorio clínico.
- El sistema depende de operación local y supervisión administrativa básica.

Explicación Operativa

Estas restricciones permiten mantener un alcance controlado y enfocado en la gestión de citas médicas y administración clínica básica, evitando complejidad funcional excesiva dentro del entorno académico.

5.6 Restricciones de Escalabilidad

La solución desarrollada no está diseñada para soportar altos niveles de concurrencia ni crecimiento masivo de información.

Restricciones de Escalabilidad Identificadas

- El sistema no soporta múltiples sedes clínicas conectadas simultáneamente.
- No se implementan mecanismos de procesamiento distribuido.
- El crecimiento masivo de registros podría afectar el rendimiento debido al almacenamiento local.
- No existen mecanismos automáticos de optimización de consultas ni indexación avanzada.
- La arquitectura actual limita la escalabilidad horizontal del sistema.

Explicación de Escalabilidad

La solución fue diseñada priorizando simplicidad arquitectónica y facilidad de mantenimiento dentro de un contexto académico. La implementación de escalabilidad empresarial requeriría migración hacia arquitecturas web distribuidas y bases de datos optimizadas.

5.7 Restricciones de Tiempo

El desarrollo del sistema se encuentra condicionado por los tiempos establecidos en el cronograma académico del curso.

Restricciones de Tiempo Identificadas

- El proyecto debe desarrollarse dentro del periodo académico correspondiente al curso de Técnicas de Programación Orientada a Objetos.
- Algunas funcionalidades avanzadas podrían implementarse en futuras versiones debido al tiempo limitado de desarrollo.
- El tiempo disponible para pruebas integrales y optimización del sistema es reducido.
- Las actividades de documentación, codificación y pruebas deben ejecutarse de manera simultánea.

Explicación Temporal

Las limitaciones de tiempo son comunes en proyectos académicos y condicionan el nivel de profundidad funcional y técnica que puede alcanzarse dentro del ciclo universitario.

5.8 Restricciones de Recursos

El proyecto presenta limitaciones relacionadas con infraestructura, personal y recursos tecnológicos disponibles.

Restricciones de Recursos Identificadas

- El sistema es desarrollado por estudiantes utilizando equipos personales.
- No se dispone de servidores empresariales ni laboratorios especializados permanentes.
- El proyecto no cuenta con personal especializado externo en ciberseguridad, arquitectura empresarial o bases de datos avanzadas.

- Los recursos tecnológicos disponibles dependen de las capacidades de hardware de cada integrante del equipo.

Explicación de Recursos

Las restricciones de recursos responden a las condiciones normales de desarrollo académico universitario, donde los proyectos se implementan utilizando herramientas y equipos accesibles para los estudiantes.

5.9 Riesgos Tecnológicos

El sistema presenta riesgos tecnológicos asociados a dependencias de software, almacenamiento local y compatibilidad tecnológica.

Riesgos Tecnológicos Identificados

- Dependencia de librerías externas como JCalendar para funcionalidades de fechas.
- Posibles incompatibilidades entre versiones de Java y ciertos componentes gráficos.
- Riesgo de pérdida de información ante fallos del equipo local.
- Riesgo de corrupción de archivos locales debido a cierres inesperados del sistema.
- Limitaciones futuras de compatibilidad con nuevas versiones del entorno Java.
- Dependencia de mantenimiento manual del código fuente y estructuras locales.

Explicación de Riesgos

La identificación de riesgos tecnológicos permite reconocer posibles limitaciones futuras del sistema y facilita la planificación de mejoras posteriores orientadas a estabilidad, seguridad y escalabilidad del software.

CAPÍTULO VI: ALTERNATIVAS DE SOLUCION

6.1 Introducción

Para el desarrollo del Sistema de Citas Médicas se analizaron diversas alternativas tecnológicas y operativas con la finalidad de determinar la solución más adecuada de acuerdo con los requerimientos funcionales, el alcance académico del proyecto, la disponibilidad de recursos y la complejidad técnica del sistema.

El análisis comparativo permitió evaluar distintas opciones considerando factores como costo, facilidad de implementación, mantenibilidad, escalabilidad, accesibilidad y compatibilidad con los objetivos del curso de Técnicas de Programación Orientada a Objetos.

6.2 Alternativa 1: Gestión Manual de Citas Médicas

Descripción

Esta alternativa consiste en administrar la información de pacientes, médicos, horarios y citas mediante registros físicos o documentos ofimáticos como hojas de cálculo y formularios impresos.

Ventajas

- Bajo costo inicial de implementación.
- No requiere conocimientos técnicos avanzados.
- No necesita infraestructura tecnológica especializada.
- Fácil adopción inicial por parte del personal administrativo.

Desventajas

- Alto riesgo de pérdida de información.
- Mayor probabilidad de errores humanos.
- Dificultad para gestionar grandes volúmenes de datos.
- Ausencia de validaciones automáticas.
- Lentitud en los procesos administrativos.
- Escasa seguridad y trazabilidad de información.

Evaluación

Aunque esta alternativa presenta costos reducidos, no garantiza eficiencia operativa ni seguridad de la información clínica, por lo que no resulta adecuada para una gestión moderna de citas médicas.

6.3 Alternativa 2: Sistema Web de Gestión Médica

Descripción

Consiste en desarrollar una plataforma web accesible desde navegadores mediante tecnologías como HTML, CSS, JavaScript y frameworks backend conectados a bases de datos remotas.

Ventajas

- Acceso remoto desde diferentes dispositivos.
- Mayor escalabilidad.
- Posibilidad de integración futura con servicios externos.
- Facilita acceso multiusuario concurrente.
- Mejor adaptabilidad para crecimiento institucional.

Desventajas

- Mayor complejidad de desarrollo.
- Requiere infraestructura de servidores.
- Incrementa los costos de implementación y mantenimiento.
- Necesita conocimientos avanzados de redes, seguridad y despliegue web.
- Mayor tiempo de desarrollo.

Evaluación

Si bien esta alternativa ofrece mayor escalabilidad y acceso remoto, supera el alcance académico y técnico definido para el presente proyecto, además de requerir recursos adicionales no disponibles durante el desarrollo.

6.4 Alternativa 3: Sistema de Escritorio en Java Swing

Descripción

La tercera alternativa corresponde al desarrollo de una aplicación de escritorio utilizando Java, Programación Orientada a Objetos y Java Swing para la construcción de interfaces gráficas.

Ventajas

- Facilidad de implementación dentro del entorno académico.
- Integración directa con conceptos de Programación Orientada a Objetos.
- Menor complejidad arquitectónica.
- Alto control sobre la lógica del sistema.
- Funcionamiento local sin necesidad de internet.
- Compatibilidad multiplataforma gracias a Java.
- Facilidad de mantenimiento modular.

Desventajas

- Acceso limitado al equipo local.
- Menor escalabilidad frente a sistemas web empresariales.
- Limitaciones para acceso remoto.
- Dependencia del entorno Java instalado.

Evaluación

Esta alternativa ofrece el mejor equilibrio entre funcionalidad, complejidad técnica, tiempo de desarrollo y cumplimiento de los objetivos académicos del curso. Además, permite aplicar de manera práctica conceptos fundamentales de Programación Orientada a Objetos, modularidad, encapsulamiento y diseño de interfaces gráficas.

6.5 Cuadro Comparativo de Alternativas

Criterio	Gestión Manual	Sistema Web	Sistema Desktop Java
Costo de implementación	Bajo	Alto	Medio
Complejidad técnica	Baja	Alta	Media
Seguridad de datos	Baja	Alta	Media
Escalabilidad	Baja	Alta	Media
Tiempo de desarrollo	Bajo	Alto	Medio
Aplicación de POO	Nula	Alta	Alta
Facilidad de mantenimiento	Baja	Media	Alta
Acceso remoto	No	Sí	No
Adecuación académica	Baja	Media	Alta

6.6 Alternativa Seleccionada

Luego del análisis comparativo realizado, se seleccionó la alternativa de desarrollo de una aplicación de escritorio utilizando Java Swing y Programación Orientada a Objetos.

La elección se fundamenta en los siguientes aspectos:

- Permite aplicar adecuadamente los conceptos del curso de Técnicas de Programación Orientada a Objetos.
- Facilita el desarrollo modular del sistema mediante separación de paquetes y clases.
- Reduce la complejidad asociada a infraestructura web y servidores.
- Optimiza el tiempo de implementación dentro del cronograma académico.
- Permite construir interfaces gráficas funcionales y amigables utilizando Java Swing.
- Garantiza estabilidad y facilidad de mantenimiento del sistema.

Asimismo, esta alternativa se adapta adecuadamente a las necesidades operativas de clínicas pequeñas y consultorios privados, proporcionando una solución funcional y escalable dentro del alcance académico establecido.

6.7 Justificación Técnica de la Solución Elegida

La implementación mediante Java Swing y arquitectura modular MVC permitió organizar el sistema en componentes independientes de modelo, vista, controlador y acceso a datos, favoreciendo la mantenibilidad y reutilización del código fuente.

Además, la utilización de Programación Orientada a Objetos permitió aplicar principios fundamentales como:

- encapsulamiento,
- herencia,
- modularidad,
- abstracción,
- reutilización de clases.

Por otro lado, el uso de herramientas como Eclipse IDE, PlantUML y GitHub permitió mejorar el control del desarrollo, la documentación técnica y la organización general del proyecto.

Finalmente, la solución seleccionada representa un equilibrio adecuado entre complejidad técnica, funcionalidad operativa y objetivos académicos, constituyendo una propuesta viable para la automatización básica de procesos administrativos en clínicas pequeñas.

CAPÍTULO VII: OBJETIVOS

7.1 Objetivo General

Desarrollar un sistema de citas médicas en Java que permita gestionar de forma eficiente pacientes, médicos, especialidades, horarios, citas y pagos, aplicando los principios de la Programación Orientada a Objetos, interfaz gráfica de usuario y acceso a base de datos relacional.

7.2 Objetivos Específicos

1. Registrar, editar, eliminar y consultar pacientes y médicos en el sistema, aplicando encapsulamiento y herencia como principios POO.
2. Gestionar especialidades médicas y horarios disponibles mediante estructuras de clases cohesivas y bien definidas.
3. Programar, cancelar y reprogramar citas médicas con validación automática de disponibilidad de horarios y conflictos de agenda.
4. Registrar pagos asociados a las atenciones y generar reportes financieros básicos.
5. Consultar el historial básico de atención de los pacientes, asegurando trazabilidad de las consultas médicas.
6. Generar reportes de citas e ingresos que faciliten la toma de decisiones administrativas.
7. Aplicar correctamente los principios de la POO encapsulamiento, herencia, polimorfismo y abstracción en el diseño modular del sistema.
8. Implementar interfaces gráficas intuitivas con Swing o JavaFX y persistencia de datos con JDBC y MySQL.

CAPÍTULO VIII: ALCANCES DEL SISTEMA

8.1 Alcance General del Sistema

El Sistema de Citas Médicas para Clínica Pequeña tiene como finalidad automatizar los procesos administrativos y operativos relacionados con la gestión de pacientes, médicos, horarios, citas médicas, pagos y reportes clínicos básicos. La solución ha sido desarrollada en el lenguaje Java aplicando el paradigma de la Programación Orientada a Objetos, utilizando interfaces gráficas Swing y persistencia de datos mediante archivos planos y estructuras organizadas por módulos.

El sistema permite centralizar la información clínica y administrativa, reduciendo errores derivados de procesos manuales y mejorando la organización operativa de la clínica. Asimismo, la solución incorpora mecanismos de validación de horarios, autenticación básica de usuarios y generación de reportes administrativos.

La arquitectura del proyecto se encuentra organizada bajo una estructura modular compuesta por paquetes de modelo, controlador, vista y datos, favoreciendo la mantenibilidad, reutilización y escalabilidad del software.

8.2 Alcance Funcional

El sistema desarrollado incluye los siguientes módulos funcionales:

8.2.1 Módulo de Gestión de Pacientes

Permite registrar, editar, eliminar y consultar pacientes mediante formularios gráficos. El sistema valida datos obligatorios como DNI, nombres y teléfonos, evitando registros incompletos o duplicados.

8.2.2 Módulo de Gestión de Médicos

Permite administrar la información de los médicos, incluyendo especialidad, horarios y disponibilidad de atención.

8.2.3 Módulo de Gestión de Especialidades

Permite registrar y listar especialidades médicas asociadas a los profesionales de salud.

8.2.4 Módulo de Gestión de Horarios

Administra horarios médicos disponibles según día, turno y disponibilidad.

8.2.5 Módulo de Gestión de Citas Médicas

Permite programar, cancelar y reprogramar citas médicas, validando automáticamente conflictos de horario entre pacientes y médicos.

8.2.6 Módulo de Gestión de Pagos

Permite registrar pagos asociados a las citas médicas y generar reportes básicos de ingresos.

8.2.7 Módulo de Historial Clínico

Permite consultar observaciones y antecedentes básicos de atención del paciente.

8.2.8 Módulo de Reportes

Genera reportes de citas, ingresos y consultas médicas para apoyo administrativo.

8.2.9 Módulo de Seguridad y Acceso

Implementa autenticación básica de usuarios mediante login y validación de credenciales.

8.3 Alcance Técnico

El sistema fue desarrollado utilizando las siguientes tecnologías y herramientas:

Tecnología	Uso
Java	Lenguaje principal
Swing	Interfaz gráfica
Eclipse IDE	Entorno de desarrollo
Git y GitHub	Control de versiones
PlantUML	Diagramas UML
JDBC	Acceso a datos
SQLite/MySQL	Persistencia
JCalendar	Manejo de fechas

8.4 Usuarios del Sistema

El sistema está orientado a los siguientes perfiles de usuario:

Usuario	Función
Recepcionista	Registro de citas y pacientes
Médico	Consulta de horarios y citas
Administrador	Gestión global del sistema
Cajero	Registro y control de pagos

8.5 Alcance Operativo

El sistema está diseñado para operar en clínicas pequeñas y consultorios privados con un número limitado de usuarios concurrentes. La solución funciona como aplicación de escritorio local y no requiere conexión permanente a internet para su funcionamiento básico.

CAPÍTULO IX: REQUERIMIENTOS FUNCIONALES

9.1 Requerimientos Funcionales

Los requerimientos funcionales definen las capacidades y comportamientos específicos que el sistema debe proveer. Siguiendo la metodología de documentación de requerimientos de sistema propuesta por Deitel y Deitel (2016), se presenta la siguiente relación de cuarenta (40) requerimientos funcionales identificados:

RF-N°	Descripción	Módulo
RF-01	El sistema debe permitir iniciar sesión con usuario y contraseña	Acceso y seguridad
RF-02	El sistema debe diferenciar roles de usuario: administrador, médico.	Acceso y seguridad
RF-03	El sistema permitirá la recuperación de contraseña mediante validación de datos personales.	Acceso y seguridad
RF-04	El sistema debe permitir registrar nuevos pacientes con nombre, apellido, DNI, fecha de nacimiento, sexo, teléfono, correo y tipo de seguro.	Pacientes
RF-05	El sistema debe permitir editar los datos de un paciente existente.	Pacientes
RF-06	El sistema debe permitir eliminar pacientes del sistema previa confirmación.	Pacientes
RF-07	El sistema debe permitir buscar pacientes por DNI	Pacientes
RF-08	El sistema debe permitir listar todos los pacientes registrados	Pacientes
RF-09	El sistema debe permitir registrar nuevos médicos con nombre, apellido, DNI, CMP, especialidad y teléfono.	Médicos
RF-10	El sistema debe permitir editar datos de médicos	Médicos
RF-11	El sistema debe permitir eliminar médicos del sistema previa confirmación.	Médicos
RF-12	El sistema debe permitir registrar especialidades médicas disponibles en la clínica.	Especialidades
RF-13	El sistema debe permitir asignar especialidad a un médico	Especialidades

RF-14	El sistema debe permitir listar especialidades disponibles	Especialidades
RF-15	El sistema debe permitir registrar horarios médicos por día de la semana y turno (mañana, tarde, noche).	Horarios
RF-16	El sistema debe permitir editar los horarios registrados de un médico.	Horarios
RF-17	El sistema debe permitir eliminar horarios médicos del sistema.	Horarios
RF-18	El sistema debe mostrar en tiempo real los horarios disponibles de cada médico al momento de programar una cita.	Horarios
RF-19	El sistema debe permitir programar citas médicas seleccionando paciente, médico, fecha y hora.	Citas
RF-20	El sistema debe validar automáticamente cruces de horario al programar citas.	Citas
RF-21	El sistema debe permitir cancelar citas programadas	Citas
RF-22	El sistema debe permitir reprogramar citas existentes a una nueva fecha u hora.	Citas
RF-23	El sistema debe permitir consultar citas filtradas por fecha.	Citas
RF-24	El sistema debe permitir consultar citas filtradas por médico.	Citas
RF-25	El sistema debe permitir consultar citas filtradas por paciente.	Citas
RF-26	El sistema debe permitir actualizar el estado de las citas (Pendiente, Atendida, Cancelada, Reprogramada).	Citas
RF-27	El sistema debe permitir listar citas canceladas con filtro de fecha y motivo	Citas
RF-28	El sistema debe permitir registrar pacientes en el módulo de urgencias sin necesidad de cita previa.	Urgencias
RF-29	El sistema debe permitir asignar un médico disponible directamente a una urgencia.	Urgencias
RF-30	El sistema debe permitir registrar el motivo de consulta por cada cita atendida.	Historial
RF-31	El sistema debe permitir registrar observaciones clínicas básicas por cada consulta atendida.	Historial
RF-32	El sistema debe permitir consultar el historial completo de atención de un paciente.	Historial
RF-33	El sistema debe permitir registrar el pago asociado a una cita atendida.	Pagos
RF-34	El sistema debe permitir consultar pagos registrados filtrados por fecha.	Pagos
RF-35	El sistema debe permitir generar reporte diario de citas programadas.	Reportes
RF-36	El sistema debe permitir generar reporte de citas por médico en un periodo determinado.	Reportes
RF-37	El sistema debe permitir generar reporte de ingresos por periodo.	Reportes

RF-38	El sistema debe mostrar mensajes de error claros ante datos inválidos o incompletos.	Validación y persistencia
RF-39	El sistema debe validar todos los campos obligatorios antes de permitir el guardado de cualquier registro.	Validación y persistencia
RF-40	El sistema debe almacenar toda la información de forma persistente en base de datos relacional.	Validación y persistencia

9.2 Requerimientos No Funcionales

- Usabilidad: La interfaz gráfica debe ser clara, intuitiva y consistente, de modo que un usuario sin conocimientos técnicos avanzados pueda operar el sistema sin dificultad.
- Modularidad: El sistema debe estar organizado en paquetes y clases con responsabilidades bien definidas, siguiendo el principio de responsabilidad única (SRP) de SOLID.
- Rendimiento: El sistema debe responder a las operaciones del usuario en un tiempo razonable, sin bloqueos perceptibles en la interfaz durante consultas a la base de datos.
- Seguridad: El acceso al sistema debe requerir autenticación obligatoria con usuario y contraseña, diferenciando los permisos según el rol del usuario.
- Mantenibilidad: El código debe estar documentado, seguir convenciones de nomenclatura Java y aplicar principios de código limpio para facilitar su mantenimiento futuro.
- Portabilidad: El sistema debe poder ejecutarse en cualquier equipo con Java instalado, independientemente del sistema operativo.

CAPÍTULO X. HISTORIAS DE USUARIO

Las historias de usuario son descripciones simples de una funcionalidad del sistema desde la perspectiva del usuario final, utilizadas en metodologías ágiles de desarrollo de software. Deitel y Deitel (2016) señalan que una historia de usuario se utiliza para describir los requisitos que un usuario desea que realice un sistema, representando sus necesidades desde su propio punto de vista. A continuación, se presentan las historias de usuario del sistema, estructuradas con los criterios de aceptación correspondientes:

ID	Título	Descripción	Criterios de Aceptación
US-01	Iniciar sesión	Como usuario del sistema (administrador, recepcionista, médico o cajero), quiero ingresar mis credenciales de usuario y contraseña, para acceder únicamente a los módulos correspondientes a mi rol sin que otros usuarios puedan ver mi información.	Credenciales correctas dan acceso al panel del rol asignado. Credenciales incorrectas muestran mensaje de error sin conceder acceso. El sistema bloquea el acceso tras tres intentos fallidos consecutivos. Requerimientos cubiertos: RF-01, RF-02
US-02	Recuperar contraseña olvidada	Como usuario registrado en el sistema, quiero recuperar mi contraseña ingresando mi DNI y correo electrónico, para poder volver a acceder al sistema sin necesidad de contactar al administrador	Datos válidos permiten establecer una nueva contraseña. Datos incorrectos muestran mensaje de error sin modificar la contraseña. Requerimientos cubiertos: RF-03
US-03	Registrar un nuevo paciente	Como recepcionista, quiero registrar los datos personales de un nuevo paciente (nombre, DNI, fecha de nacimiento, sexo, teléfono, correo y tipo de seguro), para tener su información disponible en el sistema y poder programarle citas médicas.	El registro se guarda correctamente y aparece ventana con mensaje "Paciente registrado correctamente" mostrando todos los datos ingresados. DNI duplicado o campos obligatorios vacíos muestran mensaje de error específico. Requerimientos cubiertos: RF-04
US-04	Editar datos de un paciente existente	Como recepcionista, quiero modificar los datos de un paciente ya registrado, para mantener la información actualizada cuando el paciente cambia su teléfono, dirección o tipo de seguro.	Los cambios se reflejan de forma inmediata en el sistema. No se permite guardar si hay campos obligatorios vacíos. Requerimientos cubiertos: RF-05

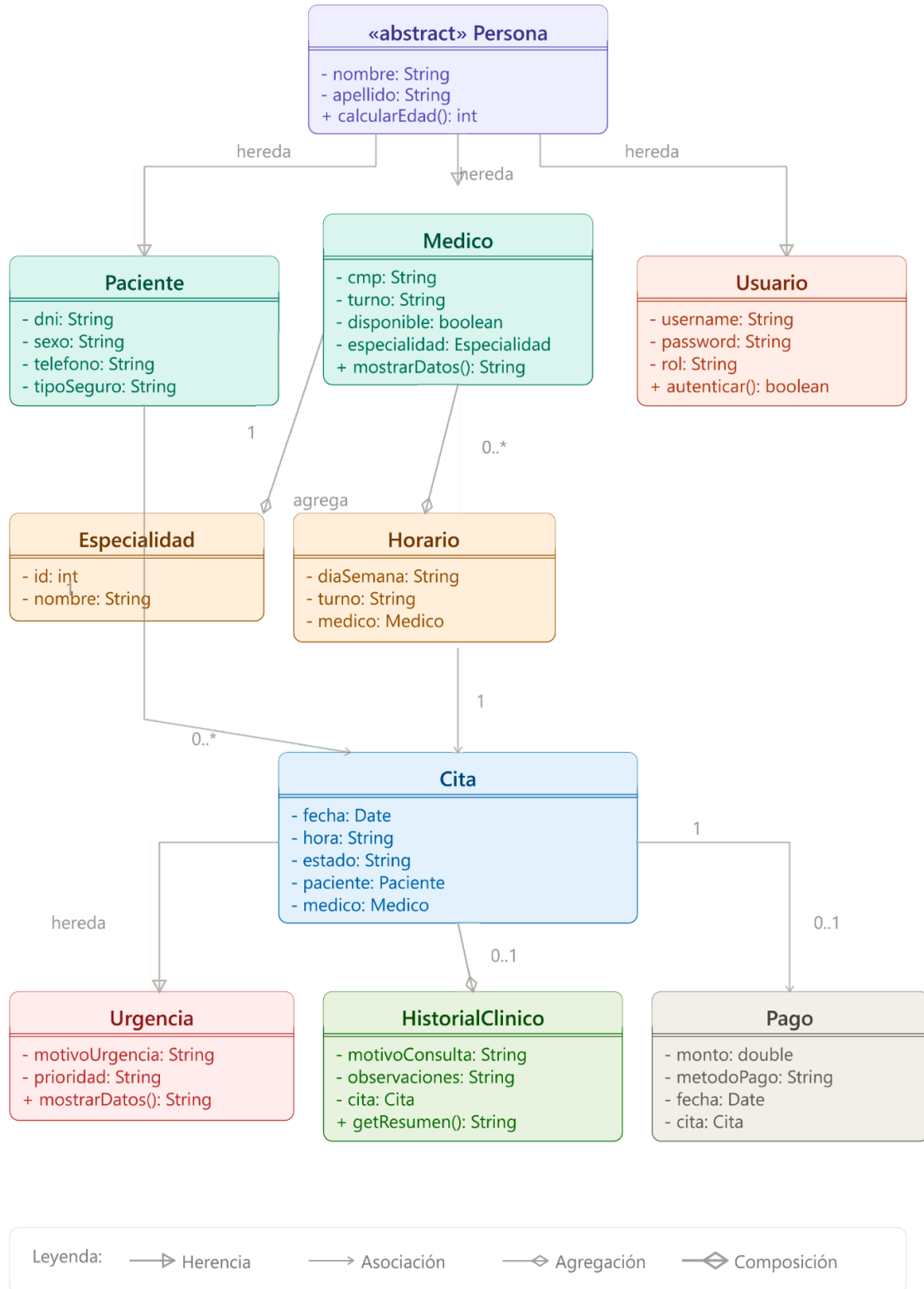
US-05	Buscar paciente por DNI	Como recepcionista, quiero buscar a un paciente escribiendo su número de DNI, para localizarlo rápidamente al momento de registrar una cita o verificar sus datos sin tener que revisar toda la lista.	La búsqueda retorna el registro del paciente si existe. Si el DNI no está registrado, se muestra mensaje "Paciente no encontrado". Requerimientos cubiertos: RF-07, RF-08
US-06	Registrar un nuevo médico	Como administrador, quiero registrar los datos de un médico (nombre, DNI, CMP, especialidad, turno y teléfono), para que pueda ser asignado a citas y urgencias dentro del sistema.	El registro se guarda correctamente y aparece ventana con mensaje "Doctor registrado correctamente" mostrando todos los datos. CMP duplicado o campos vacíos muestran mensaje de error. Requerimientos cubiertos: RF-09
US-07	Gestionar especialidades médicas	Como administrador, quiero registrar y listar las especialidades médicas disponibles en la clínica, para poder asignarlas correctamente a cada médico y facilitar la búsqueda de profesionales por área de atención.	Se puede registrar una nueva especialidad sin duplicados. La especialidad queda disponible en el selector al registrar o editar un médico. Requerimientos cubiertos: RF-12, RF-13, RF-14
US-08	Registrar y gestionar horarios médicos	Como administrador, quiero registrar los horarios de atención de cada médico indicando el día y turno, para que el sistema pueda mostrar la disponibilidad real al momento de agendar citas.	El horario se asocia al médico y queda visible al agendar citas. No se permite registrar horarios duplicados para el mismo médico, día y turno. No se puede eliminar un horario con citas asignadas. Requerimientos cubiertos: RF-15, RF-16, RF-17, RF-18
US-09	Programar una cita médica	Como recepcionista, quiero programar una cita seleccionando el paciente, el médico, la fecha y la hora, para organizar la agenda de atenciones de la clínica de forma ordenada y sin conflictos de horario.	La cita se registra con estado "Pendiente" y aparece ventana de confirmación con todos los datos. Si hay cruce de horario, el sistema muestra alerta y no permite confirmar la cita. Requerimientos cubiertos: RF-19, RF-20
US-10	Cancelar una cita médica	Como recepcionista, quiero cancelar una cita cuando el paciente lo solicita, para liberar el horario del médico y permitir que otro paciente ocupe ese espacio.	El estado de la cita cambia a "Cancelada" y el horario queda disponible. Se registra la fecha y el motivo de cancelación. Si la cita ya estaba cancelada, se muestra mensaje informativo. Requerimientos cubiertos: RF-21, RF-27
HU-11	Reprogramar una cita existente	Como recepcionista, quiero reprogramar una cita a una nueva fecha u hora cuando el paciente o el médico no puede asistir, para	Se valida disponibilidad del nuevo horario antes de confirmar la reprogramación. La cita original queda registrada con estado "Reprogramada" y se crea el nuevo registro.

		no perder la atención y mantener la organización de la agenda.	Requerimientos cubiertos: RF-22, RF-26
HU-12	Consultar la agenda de citas por fecha y médico	Como recepcionista, quiero consultar todas las citas de un día o de un médico en particular, para tener una visión completa de la agenda y orientar correctamente a los pacientes.	Se puede filtrar por fecha, por médico o por ambos. La lista muestra paciente, médico, hora, motivo y estado de cada cita. Requerimientos cubiertos: RF-23, RF-24, RF-25
HU-13	Consultar mis citas del día como médico	Como médico, quiero ver la lista de citas que tengo programadas para el día actual ordenadas por hora, para planificar mi jornada de trabajo con anticipación y atender a los pacientes puntualmente.	Se muestra la agenda del médico con nombre del paciente, hora, motivo y estado. Si no hay citas para el día, se muestra mensaje informativo. Requerimientos cubiertos: RF-24, RF-26
HU-14	Atender a un paciente de urgencias	Como médico o recepcionista, quiero registrar a un paciente que llega directamente a urgencias sin cita previa, asignando su prioridad (Alta, Media, Baja) y un médico disponible, para garantizar que reciba atención inmediata sin necesidad de pasar por el proceso regular de citas.	Se registra nombre, DNI, motivo, prioridad y médico asignado. Aparece ventana de confirmación con los datos del paciente enviado a urgencias. Solo se muestran médicos con turno activo disponibles para asignar. Requerimientos cubiertos: RF-28, RF-29
HU-15	Registrar observaciones de una consulta	Como médico, quiero registrar el motivo de consulta y mis observaciones clínicas al finalizar cada atención, para dejar constancia en el historial del paciente y que futuras consultas tengan contexto previo.	Las observaciones se guardan en el historial vinculadas a la cita, el paciente y el médico. El campo no puede quedar vacío; se muestra validación si se intenta guardar sin completar. Requerimientos cubiertos: RF-30, RF-31
HU-16	Consultar el historial de atención de un paciente	Como médico, quiero consultar el historial de consultas previas de un paciente antes de atenderlo, para conocer sus antecedentes, diagnósticos anteriores y observaciones de otros médicos que puedan ser relevantes para la nueva consulta.	El historial muestra fecha, médico, motivo, observaciones y estado de pago de cada consulta. La lista está ordenada de más reciente a más antigua. Requerimientos cubiertos: RF-32
HU-17	Registrar el pago de una atención	Como cajero, quiero registrar el pago de una cita atendida especificando el monto, el método de pago (efectivo, tarjeta o seguro) y la fecha, para llevar un control preciso de los ingresos de la clínica y que la cita quede marcada como pagada	El pago se registra correctamente y la cita queda con estado "Pagada". Si la cita no existe o ya fue pagada, se muestra mensaje de error. Requerimientos cubiertos: RF-33, RF-34

HU-18	Eliminar un registro del sistema	Como administrador, quiero eliminar pacientes o médicos que ya no están activos en la clínica, para mantener la base de datos limpia y actualizada	El sistema solicita confirmación antes de eliminar cualquier registro. No se permite eliminar un paciente o médico con citas activas asignadas. La eliminación muestra mensaje de confirmación cuando se completa correctamente. Requerimientos cubiertos: RF-06, RF-11
HU-19	Generar reportes de citas e ingresos	Como administrador, quiero generar reportes de citas por fecha o por médico, y reportes de ingresos por periodo, para supervisar el rendimiento de la clínica y tomar decisiones administrativas basadas en datos reales.	El reporte de citas muestra cantidad atendidas, canceladas y reprogramadas. El reporte de ingresos suma los pagos agrupados por método de pago. Si no hay datos para el periodo seleccionado, se muestra mensaje informativo. Requerimientos cubiertos: RF-35, RF-36, RF-37
HU-20	Validar datos antes de guardar cualquier registro	Como usuario del sistema, quiero que el sistema me avise cuando intento guardar datos incorrectos o incompletos, para evitar errores en la base de datos y mantener la integridad de la información clínica.	Cada campo con error muestra un mensaje descriptivo que indica exactamente qué debe corregir. No se permite guardar ningún registro si hay campos obligatorios vacíos o con formato inválido. Requerimientos cubiertos: RF-38, RF-39, RF-40

CAPÍTULO XI. DISEÑO DEL SISTEMA

11.1 DIAGRAMA DE CLASES



Análisis del diagrama de clases

El diagrama de clases representa la arquitectura orientada a objetos del sistema de atención de citas médicas, aplicando los principios fundamentales de la programación orientada a objetos: herencia, polimorfismo, encapsulamiento y abstracción.

11.1.1 Clase abstracta Persona

La clase Persona es el elemento central del diseño. Al ser declarada como clase abstracta, no puede instanciarse directamente, lo que obliga a que toda representación de una persona en el sistema sea a través de sus subclases concretas. Contiene los atributos comunes a cualquier persona dentro del sistema: id, nombre, apellido y dni, todos protegidos con el modificador `protected` para que las subclases puedan acceder a ellos directamente.

El método `mostrarDatos()` es declarado abstracto, lo que garantiza que cada subclase lo implemente de forma particular según su naturaleza. Esto es la base del polimorfismo en el sistema.

11.1.2 Herencia: Paciente y Medico extienden Persona

Ambas clases heredan los atributos y métodos de Persona y agregan sus propios atributos específicos:

Paciente incorpora `telefono`, `correo`, `fechaNacimiento`, `sexo`, `tipoSeguro` e `historial`, que son datos propios de la atención médica de un paciente. Implementa `mostrarDatos()` con información clínica relevante del paciente.

Medico incorpora `cmp`, `telefono`, `correo`, `turno`, `disponible` y una referencia a `Especialidad`, que son datos propios del ejercicio profesional médico. Implementa `mostrarDatos()` mostrando sus credenciales y especialidad.

Ambas clases sobrescriben el método `mostrarDatos()` con la anotación `@Override`, lo que demuestra polimorfismo de sobrescritura. Esto permite que en el sistema se pueda tratar a cualquier persona de forma genérica y al momento de ejecutar el método, Java decide qué implementación usar según el tipo real del objeto en tiempo de ejecución.

11.1.3 Clase Especialidad: asociación con Medico

Especialidad tiene una relación de asociación con Medico. Cada médico tiene asignada una especialidad que determina el área de atención. Esta clase es independiente y reutilizable, lo que permite que múltiples médicos compartan la misma especialidad sin duplicar información.

11.1.4 Clase Cita: asociación central del sistema

Cita es la clase que articula el funcionamiento principal del sistema. Asocia a un Paciente con un Medico y una Especialidad, registrando además la fecha, hora, motivo, estado y observaciones. Sus métodos cancelar(), atender() y reprogramar() encapsulan la lógica de cambio de estado, respetando el principio de responsabilidad única

11.1.5 Clase Urgencia: atención sin cita previa

Urgencia representa un caso especial del sistema donde el paciente no necesita pasar por el proceso regular de agendamiento. Se asocia directamente con un Medico disponible y registra la prioridad de atención, implementando el método asignarMedico() que permite reasignar el médico en caso necesario.

11.1.6 Clases HistorialClinico y Pago: trazabilidad

HistorialClinico depende de Cita para registrar las observaciones clínicas de cada atención, permitiendo construir el historial médico de un paciente a lo largo del tiempo.

Pago también depende de Cita para registrar el cobro asociado a cada atención, con el monto, metodoPago y fecha, lo que permite el control de ingresos de la clínica.

11.1.7 Clase Horario: gestión de disponibilidad

Horario se asocia con Medico para registrar los días y turnos de atención disponibles. Sus métodos bloquear() y liberar() permiten gestionar la disponibilidad en tiempo real al momento de agendar citas.

11.1.8. Clase Usuario — control de acceso

Usuario es independiente de la jerarquía de herencia y gestiona el acceso al sistema mediante username, password y rol. Su método autenticar() retorna un valor booleano que determina si las credenciales son válidas, separando la responsabilidad de seguridad del resto del dominio.

El diagrama demuestra una arquitectura orientada a objetos bien estructurada que aplica correctamente los cuatro pilares fundamentales. La herencia evita la duplicación de atributos

comunes entre Paciente y Medico. El polimorfismo permite tratar objetos de diferentes tipos de forma uniforme. El encapsulamiento protege los atributos mediante modificadores de acceso. La abstracción a través de la clase Persona abstracta define un contrato que todas las subclases deben cumplir. El resultado es un sistema modular, extensible y fácil de mantener, donde agregar un nuevo tipo de persona como Cajero o Administrador solo requiere crear una nueva subclase que extienda Persona sin modificar el resto del sistema.

11.2 Relaciones identificadas:

Clase origen	Tipo de relación	Clase destino
Paciente	Hereda de	Persona
Medico	Hereda de	Persona
Medico	Asociación	Especialidad
Cita	Asociación	Paciente
Cita	Asociación	Medico
HistorialClinico	Depende de	Cita
Pago	Depende de	Cita
Urgencia	Asociación	Medico
Horario	Asociación	Medico

11.3 Patrón de diseño aplicado

El sistema fue desarrollado aplicando una adaptación del patrón de arquitectura Modelo-Vista-Controlador (MVC), estructurado en paquetes independientes dentro del entorno de desarrollo Eclipse con Java Swing.

El paquete modelo contiene las clases del dominio del negocio, es decir, las entidades que representan los datos del sistema como Persona, Paciente, Medico, Cita, Urgencia, entre otras.

El paquete vista agrupa todos los formularios de la interfaz gráfica desarrollados con Swing Designer, incluyendo los JFrame, JPanel y JDialog que el usuario visualiza e interactúa directamente.

El paquete controlador actúa como intermediario entre la vista y los datos, recibiendo las solicitudes del usuario desde los formularios, aplicando las validaciones y reglas de negocio correspondientes, y retornando el resultado al formulario para que este lo muestre al usuario.

El paquete datos complementa esta arquitectura actuando como capa de persistencia en memoria mediante colecciones ArrayList, diseñado de forma que en una fase posterior pueda

ser reemplazado por consultas SQL a través de JDBC sin necesidad de modificar ni la vista ni el controlador. Esta separación de responsabilidades facilita el mantenimiento, la escalabilidad y la comprensión del sistema por parte de cualquier desarrollador que lo retome en el futuro.

CAPÍTULO XII: IMPLEMENTACIÓN

12.1 Lenguaje utilizado

El sistema fue desarrollado íntegramente en el lenguaje de programación Java, utilizando el entorno de desarrollo integrado Eclipse IDE. Para la construcción de la interfaz gráfica se empleó la librería Swing de Java junto con la herramienta Swing Designer, que permite diseñar formularios de forma visual y genera el código automáticamente. Para el componente de selección de fechas se integró la librería externa JCalendar 1.4, incorporada al proyecto mediante la carpeta lib y configurada en el Build Path de Eclipse. La gestión de datos durante esta fase del desarrollo se implementó mediante colecciones ArrayList en memoria, con una arquitectura preparada para migrar a base de datos relacional MySQL a través de JDBC en la siguiente fase del proyecto.

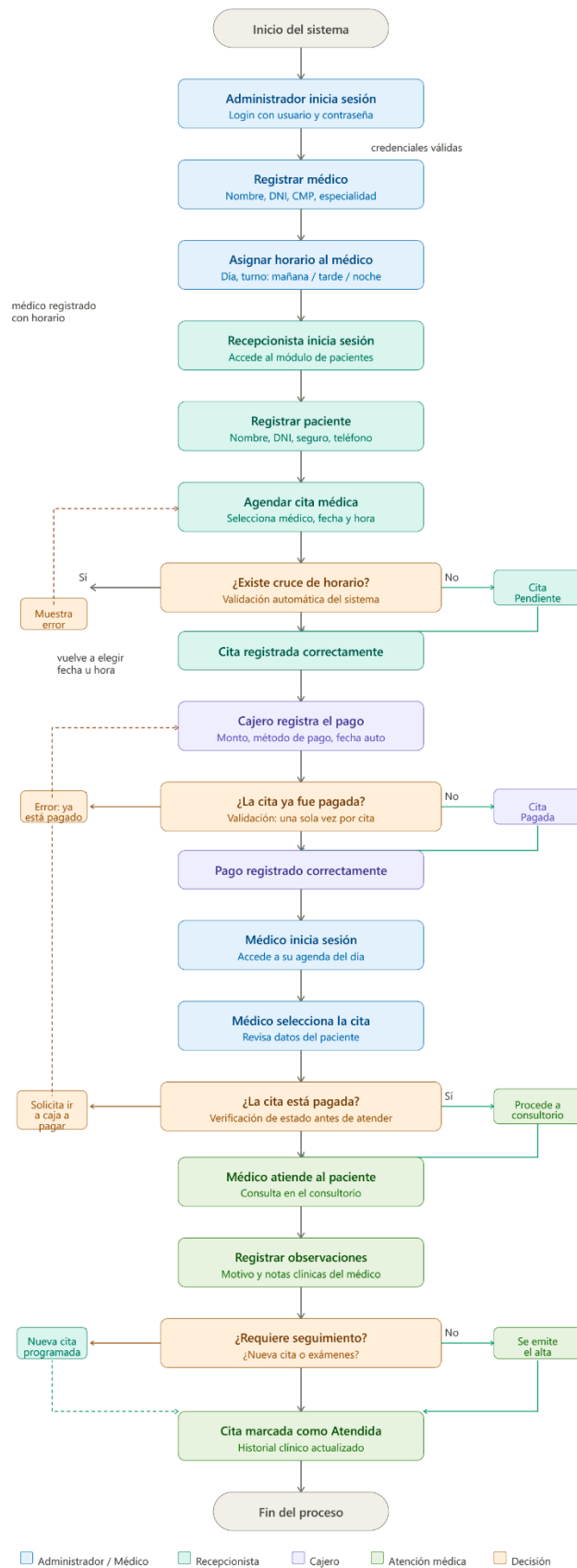
Elemento	Detalle
Lenguaje	Java SE 25
IDE	Eclipse IDE
Interfaz gráfica	Java Swing + Swing Designer
Librería adicional	JCalendar 1.4
Persistencia actual	ArrayList en memoria
Persistencia futura	MySQL + JDBC
Control de versiones	GitHub

12.2 Descripción del Sistema

El Sistema de Atención de Citas Médicas es una aplicación de escritorio desarrollada para la gestión integral de los procesos clínicos y administrativos de una clínica de salud. El sistema centraliza en una sola plataforma todas las operaciones que anteriormente se realizaban de forma manual o en registros físicos dispersos, eliminando problemas como la superposición de citas, la pérdida de información de pacientes y la falta de control sobre los procesos de atención. El sistema permite gestionar de forma ordenada el ciclo completo de atención médica, que inicia con el registro de los profesionales de salud y la configuración de sus horarios de atención, continúa con el registro de los pacientes y la programación de sus citas médicas, y finaliza con

la atención en consultorio, el registro de observaciones clínicas y la emisión del alta médica. Cada etapa del proceso está respaldada por validaciones automáticas que garantizan la integridad de la información, como la verificación de duplicados, la detección de cruces de horario y la confirmación del pago antes de proceder con la atención. El acceso al sistema está controlado mediante un módulo de autenticación que diferencia cuatro roles de usuario: el administrador, que gestiona la configuración general del sistema incluyendo médicos, especialidades y horarios; la recepcionista, que registra pacientes y coordina la agenda de citas; el cajero, que administra los pagos asociados a cada atención; y el médico, que consulta su agenda diaria, atiende a los pacientes y registra las observaciones clínicas en el historial de cada uno. Adicionalmente, el sistema cuenta con un módulo de urgencias que permite atender pacientes que llegan a la clínica sin cita previa, asignándoles un médico disponible de forma inmediata según el nivel de prioridad de su caso. El módulo de reportes proporciona al administrador una visión general del rendimiento de la clínica mediante resúmenes de citas atendidas, canceladas y reprogramadas, así como el control de ingresos desglosado por método de pago.

12.2.1 Flujo operativo del sistema



12.3 Módulos Implementados

A continuación, se describe la implementación de cada módulo funcional y su correspondencia con los requerimientos del sistema.

12.3.1 Módulo Acceso y Seguridad

Este módulo abarca los siguientes requerimientos, referentes al inicio de sesión, diferenciación de roles y recuperación de contraseña.

- **RF-01:** El sistema debe permitir iniciar sesión con usuario y contraseña
- **RF-02:** El sistema debe diferenciar roles de usuario: administrador, médico.
- **RF-03:** El sistema permitirá la recuperación de contraseña mediante validación de datos personales.

La clase central de este módulo es Usuario.java, ubicada en el paquete modelo, que extiende los atributos base de Persona.java e incorpora los campos específicos de usuario: identificador único, nombre de usuario, contraseña cifrada, rol asignado y un enlace a la entidad relacionada.

Los usuarios precargados en el sistema son:

Nº	Usuario	CONTRASEÑA	ROL
1	admin	1234	Administrador
2	repcionista	1234	Repcionista
3	medico	1234	Médico
4	cajero	1234	Cajero
5	paciente	1234	Paciente

En cuanto al requerimiento RF-03 sobre recuperación de contraseña, esta funcionalidad aún no ha sido implementada en la versión actual del sistema.

Clases involucradas:

Clase	Paquete	ROL
Usuario.java	modelo	Entidad que representa al usuario del sistema
ListaUsuarios.java	datos	Repositorio con usuarios precargados
UsuarioControlador.java	controlador	Lógica de autenticación
Login.java	vista	Formulario de inicio de sesión
MenuPrincipal.java	vista	Ventana principal con navegación lateral

12.3.2 Módulo de Gestión de Pacientes

Este módulo constituye uno de los componentes centrales de la aplicación. Permite registrar nuevos pacientes completando un formulario con datos personales y tipo de seguro, editar registros existentes, buscar pacientes en tiempo real por nombre o DNI, y eliminar registros previa confirmación. Antes de eliminar un paciente el sistema verifica que no tenga citas activas pendientes para evitar inconsistencias. El modelo de datos está representado por `Paciente.java` que hereda de la clase abstracta `Persona.java`, la cual provee los atributos comunes de nombre, apellido y DNI.

Requerimientos atendidos:

- RF-04: Registrar nuevos pacientes con nombre, apellido, DNI, fecha de nacimiento, sexo, teléfono, correo y tipo de seguro.
- RF-05: Editar los datos de un paciente existente.
- RF-06: Eliminar pacientes del sistema previa confirmación.
- RF-07: Buscar pacientes por DNI.
- RF-08: Listar todos los pacientes registrados.

Clases involucradas

Clase	Paquete	ROL
Persona.java	modelo	Clase abstracta base con atributos comunes
Paciente.java	modelo	Entidad que representa al paciente
ListaPacientes.java	datos	Repositorio en memoria con ArrayList
PacienteControlador.java	controlador	Validaciones y reglas de negocio
PacientesPanel.java	vista	Tabla y búsqueda de pacientes
RegistroPacienteDialog.java	vista	Formulario de registro y edición

Criterios de aceptación:

- El registro se guarda correctamente y aparece ventana con mensaje "Paciente registrado correctamente" mostrando todos los datos ingresados.
- DNI duplicado o campos obligatorios vacíos muestran mensaje de error específico.
- La búsqueda retorna el registro del paciente si existe. Si el DNI no está registrado se muestra mensaje "Paciente no encontrado".
- El sistema solicita confirmación antes de eliminar. No se permite eliminar un paciente con citas activas asignadas.

12.3.3 Módulo de Gestión de Médicos

Este módulo permite registrar y gestionar los profesionales médicos de la clínica. La clase Medico.java hereda de Persona.java e incorpora atributos específicos del profesional como el número de CMP, especialidad asignada, turno laboral y estado de disponibilidad. La especialidad se representa mediante un objeto Especialidad.java asociado al médico, lo que permite acceder al nombre de la especialidad directamente desde el objeto médico sin duplicar información.

Requerimientos atendidos:

- RF-09: Registrar nuevos médicos con nombre, apellido, DNI, CMP, especialidad y teléfono.
- RF-10: Editar datos de médicos.
- RF-11: Eliminar médicos del sistema previa confirmación.

Clases involucradas:

Clase	Paquete	ROL
Persona.java	modelo	Clase abstracta base con atributos comunes
Medico.java	modelo	Entidad que representa al médico
Especialidad.java	modelo	Especialidad médica asignada
ListaMedicos.java	datos	Repositorio en memoria con ArrayList
MedicoControlador.java	controlador	Validaciones y reglas de negocio
MedicosPanel.java	vista	Tabla de médicos registrados
RegistroMedicoDialog.java	vista	Formulario de registro y edición

Criterios de aceptación:

- El registro se guarda correctamente y aparece ventana con mensaje "Doctor registrado correctamente" mostrando todos los datos.
- CMP duplicado o campos vacíos muestran mensaje de error.
- No se permite eliminar un médico con citas activas asignadas.

12.3.4 Módulo de Gestión de especialidades

Este módulo gestiona el catálogo de especialidades médicas disponibles en la clínica. Las especialidades precargadas en el sistema incluyen Medicina General, Pediatría, Ginecología, Cardiología, Dermatología, Traumatología, Oftalmología, Neurología y Psiquiatría. El panel permite registrar nuevas especialidades y listar las existentes. La asignación de especialidad a un médico se realiza desde el formulario de registro de médicos mediante un selector desplegable que se alimenta dinámicamente desde ListaEspecialidades.java.

Requerimientos atendidos:

- RF-12: Registrar especialidades médicas disponibles en la clínica.
- RF-13: Asignar especialidad a un médico.
- RF-14: Listar especialidades disponibles.

Clases involucradas:

Clase	Paquete	ROL
Especialidad.java	modelo	Entidad que representa la especialidad
ListaEspecialidades.java	datos	Repositorio con especialidades precargadas
EspecialidadesPanel.java	vista	Panel de gestión de especialidades
RegistroMedicoDialog.java	vista	Selector de especialidad al registrar médico

12.3.5 Módulo de Gestión de Horarios

Este módulo gestiona los horarios de atención de cada médico. La clase Horario.java relaciona un médico con un día de la semana y un turno laboral. Al momento de programar una cita, el sistema consulta los horarios registrados para mostrar únicamente los médicos disponibles en el día y hora seleccionados.

Requerimientos atendidos:

- RF-15: Registrar horarios médicos por día de la semana y turno.
- RF-16: Editar los horarios registrados de un médico.
- RF-17: Eliminar horarios médicos del sistema.
- RF-18: Mostrar en tiempo real los horarios disponibles al programar una cita.

Clases involucradas:

Clase	Paquete	ROL
Especialidad.java	modelo	Entidad que representa la especialidad
ListaEspecialidades.java	datos	Repositorio con especialidades precargadas
EspecialidadesPanel.java	vista	Panel de gestión de especialidades
RegistroMedicoDialog.java	vista	Selector de especialidad al registrar médico

12.3.6 Módulo de Gestión de Citas Médicas

Este módulo constituye el núcleo funcional del sistema. La clase Cita.java relaciona un paciente con un médico registrando fecha, hora, motivo, estado y observaciones. El sistema valida automáticamente los cruces de horario antes de confirmar una cita: si el médico ya tiene una cita en ese horario se muestra un mensaje de error y no se permite el registro. El flujo de estados de una cita es el siguiente: Pendiente → Pagado → Atendida, con posibilidad de pasar a Cancelada o Reprogramada en cualquier momento.

Requerimientos atendidos:

- RF-19: Programar citas médicas seleccionando paciente, médico, fecha y hora.
- RF-20: Validar automáticamente cruces de horario al programar citas.
- RF-21: Cancelar citas programadas.
- RF-22: Reprogramar citas existentes a una nueva fecha u hora.
- RF-23: Consultar citas filtradas por fecha.
- RF-24: Consultar citas filtradas por médico.
- RF-25: Consultar citas filtradas por paciente.
- RF-26: Actualizar el estado de las citas.
- RF-27: Listar citas canceladas con filtro de fecha y motivo.

Clases involucradas:

Clase	Paquete	ROL
Cita.java	modelo	Entidad central del módulo
ListaCitas.java	datos	Repositorio y validación de conflictos
CitaControlador.java	controlador	Lógica de agendamiento y estados
AgendarCitaPanel.java	vista	Formulario de programación de cita
GestionCitasPanel.java	vista	Tabla con gestión de estados
ReprogramarCitaDialog.java	vista	Diálogo de reprogramación

12.3.7 Módulo de Urgencias Médicas

Este módulo permite registrar pacientes que requieren atención inmediata sin pasar por el proceso regular de agendamiento. La clase Urgencia.java almacena el motivo, nivel de prioridad (Alta, Media, Baja) y el médico asignado. A diferencia de las citas regulares, las urgencias no requieren programación previa y pueden registrarse en cualquier momento desde el panel de urgencias.

Requerimientos atendidos:

- RF-28: Registrar pacientes en urgencias sin necesidad de cita previa.
- RF-29: Asignar un médico disponible directamente a una urgencia.

Clases involucradas:

Clase	Paquete	ROL
Urgencia.java	modelo	Entidad de atención de urgencia
ListaUrgencias.java	datos	Repositorio de urgencias en memoria
UrgenciaControlador.java	controlador	Lógica de registro y alta médica
UrgenciasPanel.java	vista	Panel de urgencias activas
RegistroUrgenciaDialog.java	vista	Formulario de registro de urgencia

Criterios de aceptación:

- Se registra nombre, DNI, motivo, prioridad y médico asignado.
- Aparece ventana de confirmación con los datos del paciente enviado a urgencias.
- Solo se muestran médicos con turno activo disponibles para asignar.

12.3.8 Módulo de Historial Clínico

Este módulo permite almacenar y consultar el registro de atenciones médicas realizadas a cada paciente. El médico puede registrar observaciones clínicas al finalizar cada consulta, las cuales quedan vinculadas a la cita correspondiente. El panel de historial permite filtrar las atenciones por paciente y visualizar la información completa de cada consulta incluyendo médico, fecha, motivo y observaciones.

Requerimientos atendidos:

- RF-30: Registrar el motivo de consulta por cada cita atendida.
- RF-31: Registrar observaciones clínicas básicas por cada consulta atendida.
- RF-32: Consultar el historial completo de atención de un paciente.

Criterios de aceptación:

- Las observaciones se guardan en el historial vinculadas a la cita, el paciente y el médico.
- El campo no puede quedar vacío; se muestra validación si se intenta guardar sin completar.
- El historial muestra fecha, médico, motivo, observaciones y estado de cada consulta.

Clases involucradas:

Clase	Paquete	ROL
HistorialClinico.java	modelo	Entidad de registro clínico
CitaControlador.java	controlador	Recuperación de citas atendidas
HistorialPanel.java	vista	Panel de consulta de historial
ObservacionesDialog.java	vista	Diálogo de registro de observaciones

12.3.9 Módulo de Gestión de Pagos

Este módulo permite al cajero registrar los cobros asociados a citas atendidas. La clase Pago.java almacena el monto, método de pago (Efectivo, Tarjeta, EsSalud, SIS, SOAT), la fecha tomada automáticamente del sistema y la referencia a la cita que origina el pago. El sistema valida que una cita no pueda ser pagada más de una vez y que solo puedan registrarse pagos de citas con estado Atendida.

Requerimientos atendidos:

- RF-33: Registrar el pago asociado a una cita atendida.
- RF-34: Consultar pagos registrados filtrados por fecha.

Criterios de aceptación:

- El pago se registra correctamente y la cita queda con estado "Pagada".
- Si la cita no existe o ya fue pagada se muestra mensaje de error.

Clases involucradas:

Clase	Paquete	ROL
Pago.java	modelo	Entidad de registro de pago
ListaPagos.java	datos	Repositorio de pagos en memoria
PagoControlador.java	controlador	Lógica de registro y validación
PagosPanel.java	vista	Panel de pagos registrados
RegistroPagoDialog.java	vista	Formulario de registro de pago

12.3.10 Módulo de Reportes

Este módulo proporciona al administrador una visión general del rendimiento de la clínica. Presenta tres secciones: resumen estadístico con tarjetas numéricas, tabla de citas atendidas agrupadas por médico con totales de ingresos, y tabla de ingresos desglosados por método de pago. El DashboardPanel.java complementa este módulo con un resumen visual rápido disponible desde el inicio del sistema.

Requerimientos atendidos:

- RF-35: Generar reporte diario de citas programadas.
- RF-36: Generar reporte de citas por médico en un periodo determinado.
- RF-37: Generar reporte de ingresos por periodo.

Criterios de aceptación:

- El reporte de citas muestra cantidad atendidas, canceladas y reprogramadas.
- El reporte de ingresos suma los pagos agrupados por método de pago.
- Si no hay datos para el periodo seleccionado se muestra mensaje informativo.

Clases involucradas:

Clase	Paquete	ROL
ReportesPanel.java	vista	Panel principal de reportes
DashboardPanel.java	vista	Panel de resumen del sistema
CitaControlador.java	controlador	Datos de citas para reportes
PagoControlador.java	controlador	Datos de ingresos para reportes

12.3.11 Módulo de Validación y Persistencia

La validación de campos obligatorios se implementa en todos los controladores del sistema antes de proceder con cualquier operación de guardado. Los mensajes de error se muestran mediante JOptionPane indicando específicamente qué campo falta o qué formato es incorrecto. Respecto al RF-40, el sistema utiliza almacenamiento en memoria mediante ArrayList en los paquetes de datos. Esta arquitectura está diseñada para que en una versión futura las clases del paquete datos sean reemplazadas por implementaciones JDBC con MySQL sin necesidad de modificar ni la vista ni el controlador.

Requerimientos atendidos:

- RF-38: Mostrar mensajes de error claros ante datos inválidos o incompletos.
- RF-39: Validar todos los campos obligatorios antes de permitir el guardado.
- RF-40: Almacenar la información de forma persistente.

Criterios de aceptación:

- Cada campo con error muestra un mensaje descriptivo que indica exactamente qué debe corregir.
- No se permite guardar ningún registro si hay campos obligatorios vacíos o con formato inválido.

Clases involucradas

Clase	Paquete	ROL
PacienteControlador.java	controlador	Validaciones de registro y edición de pacientes
MedicoControlador.java	controlador	Validaciones de registro y edición de médicos
CitaControlador.java	controlador	Validaciones de agendamiento y estados de citas
UrgenciaControlador.java	controlador	Validaciones de registro de urgencias
PagoControlador.java	controlador	Validaciones de registro de pagos

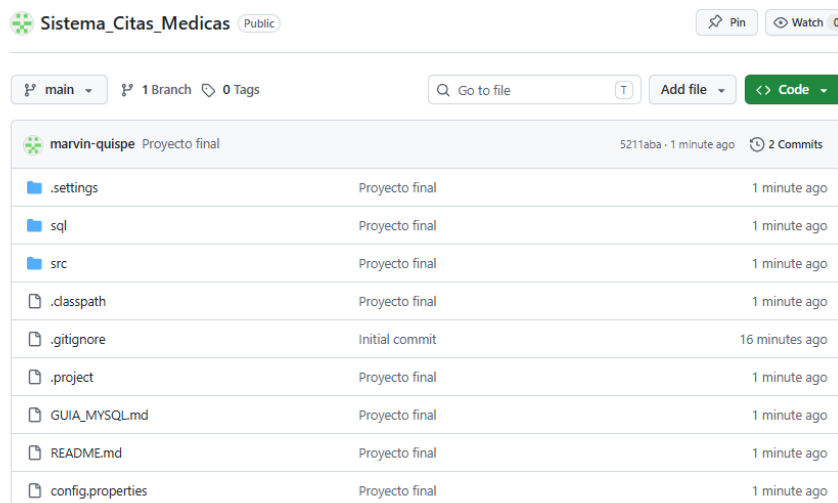
ListaPacientes.java	datos	Persistencia en memoria de pacientes
ListaMedicos.java	datos	Persistencia en memoria de médicos
ListaCitas.java	datos	Persistencia en memoria de citas
ListaUrgencias.java	datos	Persistencia en memoria de urgencias
ListaPagos.java	datos	Persistencia en memoria de pagos
ListaHorarios.java	datos	Persistencia en memoria de horarios
ListaEspecialidades.java	datos	Persistencia en memoria de especialidades

CAPÍTULO XIII: EVIDENCIA DEL SOFTWARE

13.1 Link de Github

Se adjunta el link al repositorio correspondiente al proyecto final del sistema de atención de citas médicas para clínica:

https://github.com/marvin-quispe/Sistema_Citas_Medicas.git



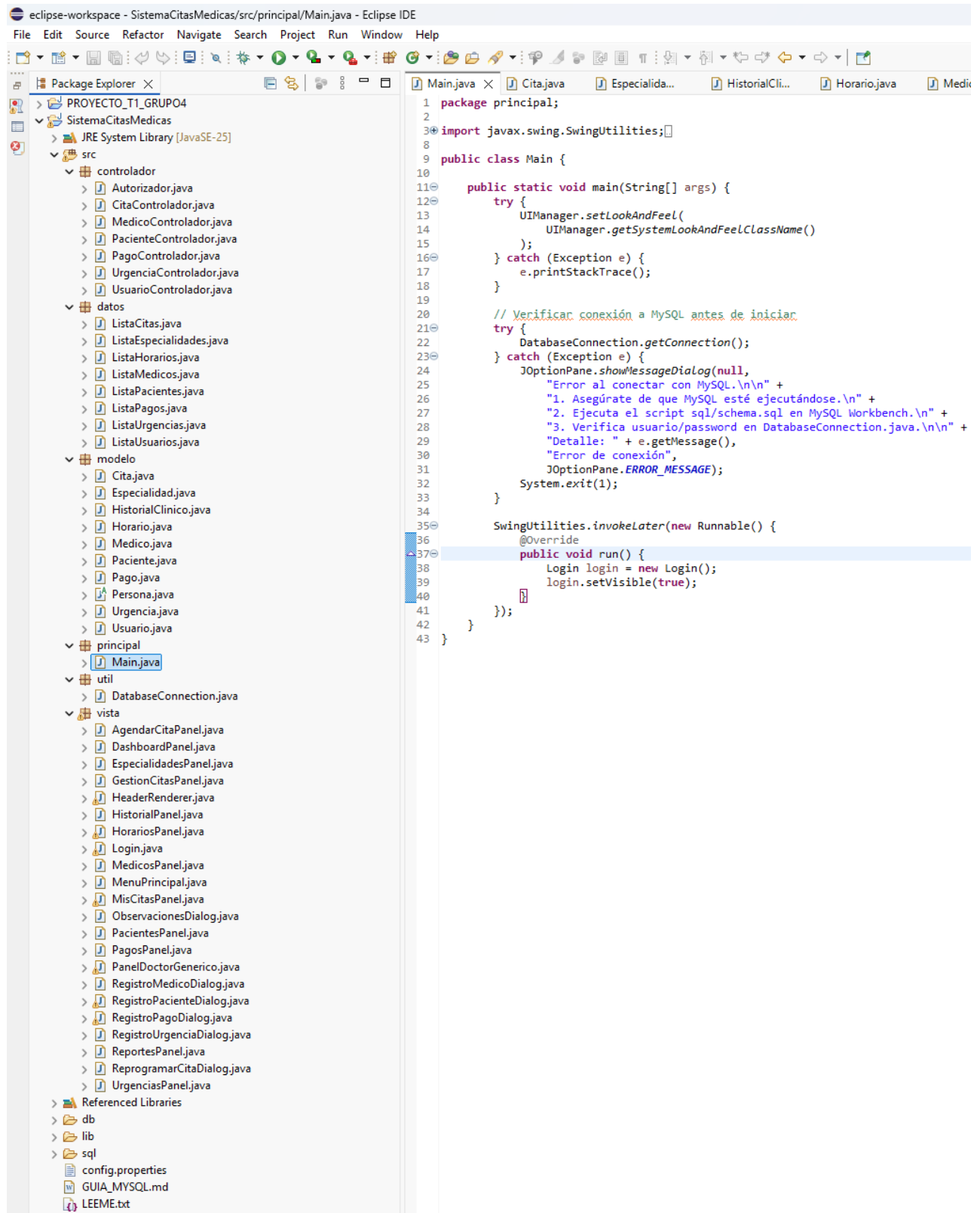
- Preparando los archivos y carga.

```
MARVINHARLEY@DESKTOP-28CUSLC MINGW64 ~/git/Sistema_Citas_Medicas (main)
$ git push
Enumerating objects: 71, done.
Counting objects: 100% (71/71), done.
Delta compression using up to 20 threads
Compressing objects: 100% (69/69), done.
Writing objects: 100% (70/70), 59.16 KiB | 5.92 MiB/s, done.
Total 70 (delta 23), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (23/23), done.
To https://github.com/marvin-quispe/Sistema_Citas_Medicas.git
39c88ac..5211aba main -> main
```

13.2 Estructura General del proyecto

El proyecto fue desarrollado en Eclipse IDE y organizado en seis paquetes principales que separan las responsabilidades del sistema siguiendo el patrón MVC.

La estructura de archivos del proyecto es la siguiente:

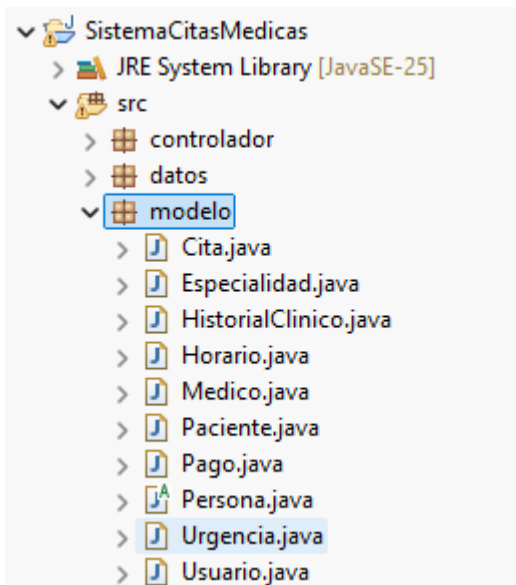


13.3 Implementación de paquetes

13.3.1 Paquete modelo

El paquete modelo constituye el núcleo conceptual del sistema. Contiene todas las clases que representan las entidades del mundo real de una clínica médica: personas, citas, pagos, historiales y urgencias. Es el paquete de mayor profundidad en términos de diseño orientado a objetos, ya que concentra los cuatro pilares de la programación orientada a objetos: abstracción, herencia, encapsulamiento y polimorfismo.

Pilar POO evidenciado: Abstracción · Herencia · Encapsulamiento · Polimorfismo



Clase	Descripción
Persona.java	Clase abstracta base. Define los atributos comunes (id, nombre, apellido, dni) y declara el método abstracto mostrarDatos(), que obliga a todas las subclases a implementar su propia versión.
Paciente.java	Hereda de Persona. Agrega los atributos propios del paciente: teléfono, correo, fecha de nacimiento, sexo y tipo de seguro. Implementa mostrarDatos() con la información clínica del paciente.
Medico.java	Hereda de Persona. Agrega CMP, turno, disponibilidad y una referencia a un objeto Especialidad. Implementa mostrarDatos() mostrando los datos profesionales del médico.

Clase	Descripción
Usuario.java	Modela la identidad de acceso al sistema. Encapsula username, password y rol. Expone el método autenticar() para validar credenciales y métodos de rol como esAdmin() y esMedico().
Cita.java	Entidad central del sistema. Asocia un Paciente y un Médico con fecha, hora y estado. Contiene métodos de ciclo de vida: cancelar(), pagar(), atender() y reprogramar().
Urgencia.java	Modela atenciones sin cita previa. Contiene motivo, nivel de prioridad (Alta/Media/Baja) y referencia a un Médico disponible. Incluye los métodos darAlta() y asignarMedico().
HistorialClinico.java	Registra el motivo de consulta y las observaciones clínicas asociadas a una Cita ya atendida. Permite la trazabilidad de cada atención médica.
Pago.java	Registra el monto, método de pago y fecha de cada transacción vinculada a una Cita. Los métodos de pago soportados son: Efectivo, Tarjeta, EsSalud, SIS y SOAT.
Especialidad.java	Representa una especialidad médica (ej. Cardiología, Pediatría). Implementa equals() y hashCode() basados en el identificador, lo que permite que el JComboBox seleccione correctamente al editar un médico.
Horario.java	Asocia a un Médico con un día de la semana y un turno (Mañana/Tarde/Noche). Incluye los métodos bloquear() y liberar() para gestionar la disponibilidad.

13.3.1.1 Persona.java – Abstracción y base de herencia

Persona es una clase abstracta que no puede instanciarse directamente. Su propósito es definir los atributos y comportamientos comunes a todas las personas del sistema (pacientes, médicos y usuarios), garantizando que cada subclase provea su propia implementación del método mostrarDatos().

Abstracción: La clase declara mostrarDatos() como método abstracto, lo que significa que no tiene cuerpo en Persona. Cada subclase está obligada por el compilador a implementarlo con su propio comportamiento.

Encapsulamiento: Los atributos id, nombre, apellido y dni están declarados como protected, lo que permite que las subclases los hereden y accedan directamente, pero los oculta de clases externas al paquete.

```

6
7 public abstract class Persona {
8     protected int id;
9     protected String nombre;
10    protected String apellido;
11    protected String dni;
12
13+   public Persona(){}
14
15
16
17+   public Persona(int id, String nombre, String apellido, String dni) {}
18
19
20
21
22
23
24    // Método abstracto - obliga a Paciente y Medico a implementarlo
25    // Esto demuestra polimorfismo
26    public abstract void mostrarDatos();
27
28    // Getters y Setters
29+   public int getId() {}
30
31
32
33+   public void setId(int id) {}
34
35
36
37+   public String getNombre() {}
38
39
40
41+   public void setNombre(String nombre) {}
42
43
44
45+   public String getApellido() {}
46
47
48
49+   public void setApellido(String apellido) {}
50
51
52
53+   public String getDni() {}
54
55
56
57+   public void setDni(String dni) {}
58
59
60
61    // Calcula la edad a partir de la fecha de nacimiento (formato yyyy-MM-dd)
62    // Se usa como campo informativo en las vistas, no como dato persistente
63+   public static int calcularEdad(String fechaNacimiento) {}
64
65
66
67
68
69
70
71
72    // Método concreto compartido por todas las subclases
73-   public String getNombreCompleto() {
74        return nombre + " " + apellido;
75    }
76 }
77
  
```

13.3.1.2 Paciente.java – Herencia y polimorfismo

Paciente extiende Persona e incorpora los datos propios de un paciente clínico. Al heredar de Persona, reutiliza los atributos base (nombre, apellido, dni) sin necesidad de redeclararlos, y cumple el contrato abstracto implementando mostrarDatos() con la información específica del paciente.

Herencia: La palabra clave `extends` Persona establece la relación es-un. El constructor llama a `super(id, nombre, apellido, dni)` para inicializar los atributos heredados.

Polimorfismo: La anotación `@Override` indica que `mostrarDatos()` reemplaza la versión abstracta de Persona. En tiempo de ejecución, si se invoca `mostrarDatos()` sobre una referencia de tipo Persona que apunta a un objeto Paciente, se ejecutará la versión de Paciente.

```

1 package modelo;
2
3 public class Paciente extends Persona {
4
5     private String telefono;
6     private String correo;
7     private String fechaNacimiento; // formato: yyyy-MM-dd
8     private String sexo;           // "Masculino" o "Femenino"
9     private String tipoSeguro;     // SIS, EsSalud, Particular, SOAT
10
11     public Paciente() {
12     }
13
14     public Paciente(int id, String nombre, String apellido, String dni,
15                     String telefono, String correo, String fechaNacimiento,
16                     String sexo, String tipoSeguro) {
17         super(id, nombre, apellido, dni);
18         this.telefono = telefono;
19         this.correo = correo;
20         this.fechaNacimiento = fechaNacimiento;
21         this.sexo = sexo;
22         this.tipoSeguro = tipoSeguro;
23     }
24
25     // Polimorfismo: sobrescribe el método abstracto de Persona
26     @Override
27     public void mostrarDatos() {
28         System.out.println("=== DATOS DEL PACIENTE ===");
29         System.out.println("ID          : " + id);
30         System.out.println("Nombre     : " + getNombreCompleto());
31         System.out.println("DNI        : " + dni);
32         System.out.println("Teléfono   : " + telefono);
33         System.out.println("Correo     : " + correo);
34         System.out.println("F. Nac.    : " + fechaNacimiento);
35         System.out.println("Sexo       : " + sexo);
36         System.out.println("Seguro     : " + tipoSeguro);
37     }
38

```

13.3.1.3 Medico.java – Herencia y agregación

Medico también extiende Persona y aplica el mismo patrón de herencia. A diferencia de Paciente, agrega atributos propios de un profesional de salud: número de CMP (Colegio Médico del Perú), turno de trabajo, disponibilidad y una referencia a un objeto de tipo Especialidad.

Herencia: Igual que Paciente, usa super() para inicializar los atributos comunes de Persona.

Polimorfismo: Implementa @Override de mostrarDatos() mostrando los datos profesionales del médico, incluyendo su especialidad si tiene una asignada.

Encapsulamiento: El atributo disponible es de tipo boolean y se accede exclusivamente a través de isDisponible() y setDisponible(), sin exposición directa del campo.

```

1  package modelo;
2
3  public class Medico extends Persona {
4
5      private String cmp;          // Colegio Médico del Perú
6      private String telefono;
7      private String correo;
8      private String turno;       // Mañana, Tarde, Noche
9      private boolean disponible;
10     private Especialidad especialidad;
11
12     public Medico() {
13     }
14
15     public Medico(int id, String nombre, String apellido, String dni,
16                   String cmp, String telefono, String correo,
17                   String turno, boolean disponible, Especialidad especialidad) {
18         super(id, nombre, apellido, dni);
19         this.cmp = cmp;
20         this.telefono = telefono;
21         this.correo = correo;
22         this.turno = turno;
23         this.disponible = disponible;
24         this.especialidad = especialidad;
25     }
26
27     // Polimorfismo: sobrescribe el método abstracto de Persona
28     @Override
29     public void mostrarDatos() {
30         System.out.println("=== DATOS DEL MÉDICO ===");
31         System.out.println("ID           : " + id);
32         System.out.println("Nombre      : " + getNombreCompleto());
33         System.out.println("DNI        : " + dni);
34         System.out.println("CMP        : " + cmp);
35         System.out.println("Especialidad : " + (especialidad != null ? especialidad.getNombre() : "No asignada"));
36         System.out.println("Turno       : " + turno);
37         System.out.println("Teléfono    : " + telefono);
38         System.out.println("Disponible  : " + (disponible ? "Sí" : "No"));
39     }
40

```

13.3.1.4 Usuario.java – Encapsulamiento y control de roles

Usuario modela la identidad de acceso al sistema. A diferencia de Paciente y Medico, no hereda de Persona porque un usuario es una entidad de autenticación y no necesariamente una persona registrada clínicamente. Todos sus atributos son privados y el acceso se realiza exclusivamente a través de métodos.

Encapsulamiento: Los atributos username y password son privados. El método autenticar() valida las credenciales sin exponer la contraseña internamente a otras clases. Los métodos esAdmin(), esMedico() y esPaciente() encapsulan la lógica de verificación de rol.

```

1 package modelo;
2
3 public class Usuario {
4
5     private int id;
6     private String username;
7     private String password;
8     private String rol;
9     private String nombre;
10    private int entidadId;
11
12    public static final String ADMIN = "Administrador";
13    public static final String RECEPCIONISTA = "Recepcionista";
14    public static final String MEDICO = "Médico";
15    public static final String CAJERO = "Cajero";
16    public static final String PACIENTE = "Paciente";
17
18    public Usuario() {
19    }
20
21    public Usuario(int id, String username, String password, String rol, String nombre) {
22        this.id = id;
23        this.username = username;
24        this.password = password;
25        this.rol = rol;
26        this.nombre = nombre;
27        this.entidadId = 0;
28    }
29
30    public Usuario(int id, String username, String password, String rol, String nombre, int entidadId) {
31        this.id = id;
32        this.username = username;
33        this.password = password;
34        this.rol = rol;
35        this.nombre = nombre;
36        this.entidadId = entidadId;
37    }
38
39    public boolean autenticar(String user, String pass) {
40        return this.username.equals(user) && this.password.equals(pass);
41    }
42
43    ..
  
```

13.3.1.5 Cita.java – Asociación y ciclo de vida

Cita es la entidad central del sistema. Asocia un Paciente y un Médico con una fecha, hora y estado. Los estados posibles son: Pendiente, Pagado, Atendida, Cancelada y Reprogramada. La clase expone métodos de ciclo de vida que cambian el estado de manera controlada.

Encapsulamiento: Todos los atributos son privados. El cambio de estado no se realiza asignando directamente el campo estado, sino a través de métodos específicos: cancelar(), pagar(), atender() y reprogramar(), que garantizan transiciones de estado válidas.

```

1 package modelo;
2
3 public class Cita {
4
5     private int id;
6     private Paciente paciente;
7     private Medico medico;
8     private Especialidad especialidad;
9     private String fecha;        // formato: yyyy-MM-dd
10    private String hora;         // formato: HH:mm
11    private String motivo;
12    private String estado;        // Pendiente, Pagado, Atendida, Cancelada, Reprogramada
13    private String observaciones;
14
15    public Cita() {
16    }
17
18    public Cita(int id, Paciente paciente, Medico medico,
19                Especialidad especialidad, String fecha, String hora,
20                String motivo, String estado) {
21        this.id = id;
22        this.paciente = paciente;
23        this.medico = medico;
24        this.especialidad = especialidad;
25        this.fecha = fecha;
26        this.hora = hora;
27        this.motivo = motivo;
28        this.estado = estado;
29        this.observaciones = "";
30    }
31
32    public void cancelar() {
33        this.estado = "Cancelada";
34    }
35
36    public void pagar() {
37        this.estado = "Pagado";
38    }
39
40    public void atender() {
41        this.estado = "Atendida";
42    }
43
44    public void reprogramar(String nuevaFecha, String nuevaHora) {
45        this.fecha = nuevaFecha;
46        this.hora = nuevaHora;
47        this.estado = "Reprogramada";
48    }

```

13.3.1.6 Urgencia.java – Atención sin cita previa

Urgencia modela casos de atención inmediata. Contiene los campos motivo, prioridad (Alta/Media/Baja), estado y una referencia a un Médico disponible. A diferencia de las citas regulares, no requiere un Paciente previamente registrado en el sistema, por lo que almacena el nombre y DNI directamente como strings.

Método relevante: darAlta() cambia el estado a "Alta médica" de manera controlada. asignarMedico() permite reasignar el médico de urgencia en cualquier momento.

```

1 package modelo;
2
3 public class Urgencia {
4
5     private int id;
6     private String nombre; // puede no estar registrado como Paciente
7     private String dni;
8     private String motivo;
9     private String prioridad; // Alta, Media, Baja
10    private Medico medico;
11    private String estado; // En atención, Alta médica
12    private String horaIngreso; // formato: HH:mm
13
14    public Urgencia() {
15    }
16
17    public Urgencia(int id, String nombre, String dni, String motivo,
18                    String prioridad, Medico medico,
19                    String estado, String horaIngreso) {
20        this.id = id;
21        this.nombre = nombre;
22        this.dni = dni;
23        this.motivo = motivo;
24        this.prioridad = prioridad;
25        this.medico = medico;
26        this.estado = estado;
27        this.horaIngreso = horaIngreso;
28    }
29
30    public void darAlta() {
31        this.estado = "Alta médica";
32    }
33
34    public void asignarMedico(Medico medico) {
35        this.medico = medico;
36    }
37

```

13.3.1.7 HistorialClinico.java – Trazabilidad clínica

Registra el motivo de consulta y las observaciones del médico para cada Cita atendida. La relación con Cita es de composición: un historial clínico no tiene sentido sin la cita que le da origen.

```

1 package modelo;
2
3 public class HistorialClinico {
4
5     private int id;
6     private Cita cita;
7     private String observaciones;
8     private String fecha; // fecha del registro
9
10    public HistorialClinico() {
11    }
12
13    public HistorialClinico(int id, Cita cita, String observaciones, String fecha) {
14        this.id = id;
15        this.cita = cita;
16        this.observaciones = observaciones;
17        this.fecha = fecha;
18    }
19

```


13.3.1.8 Pago.java – Registro de transacciones

Almacena el monto, método de pago y fecha de cada cobro vinculado a una Cita. La referencia a Cita permite tener trazabilidad completa del flujo: programación → pago → atención.

```

1 package modelo;
2
3 public class Pago {
4
5     private int id;
6     private Cita cita;
7     private double monto;
8     private String metodoPago; // Efectivo, Tarjeta, EsSalud, SIS, SOAT
9     private String fecha;      // formato: yyyy-MM-dd
10
11     public Pago() {
12     }
13
14     public Pago(int id, Cita cita, double monto,
15                 String metodoPago, String fecha) {
16         this.id = id;
17         this.cita = cita;
18         this.monto = monto;
19         this.metodoPago = metodoPago;
20         this.fecha = fecha;
21     }

```

13.3.1.9 Especialidad.java – Soporte a selección en formularios

Representa una especialidad médica. Su detalle más relevante de implementación es que sobrescribe equals() y hashCode() basados en el identificador de la especialidad. Esto es necesario para que el componente JComboBox de la interfaz gráfica pueda localizar y preseleccionar el objeto correcto al abrir el formulario de edición de un médico existente.

```

1 package modelo;
2
3 public class Especialidad {
4
5     private int id;
6     private String nombre;
7     private String descripcion;
8
9     public Especialidad() {
10    }
11
12     public Especialidad(int id, String nombre, String descripcion) {
13         this.id = id;
14         this.nombre = nombre;
15         this.descripcion = descripcion;
16     }
17
18     @Override
19     public String toString() {
20         return nombre;
21     }
22
23     @Override
24     public boolean equals(Object obj) {
25         if (this == obj) return true;
26         if (obj == null || getClass() != obj.getClass()) return false;
27         Especialidad otra = (Especialidad) obj;
28         return this.id == otra.id;
29     }
30
31     @Override
32     public int hashCode() {
33         return Integer.hashCode(id);
34     }

```

13.3.1.10 Horario.java – Disponibilidad del médico

Asocia a un Médico con un día de la semana y un turno. Los métodos bloquear() y liberar() permiten gestionar la disponibilidad de forma semántica, sin acceso directo al atributo booleano disponible.

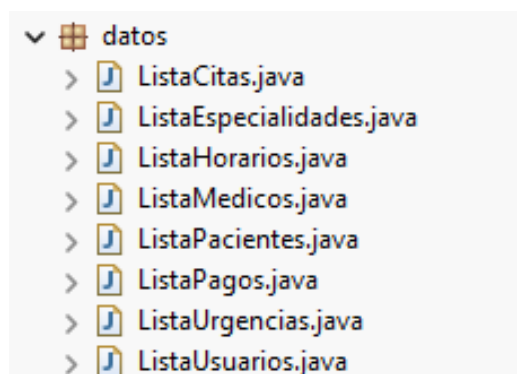
```

1 package modelo;
2
3 public class Horario {
4
5     private int id;
6     private Medico medico;
7     private String diaSemana; // Lunes, Martes, etc.
8     private String turno; // Mañana, Tarde, Noche
9     private boolean disponible;
10
11     public Horario() {
12     }
13
14     public Horario(int id, Medico medico, String diaSemana,
15                     String turno, boolean disponible) {
16         this.id = id;
17         this.medico = medico;
18         this.diaSemana = diaSemana;
19         this.turno = turno;
20         this.disponible = disponible;
21     }
22
23     public void bloquear() {
24         this.disponible = false;
25     }
26
27     public void liberar() {
28         this.disponible = true;
29     }

```

13.3.2 Paquete datos

El paquete datos actúa como capa de persistencia en memoria del sistema. Cada clase de este paquete administra una colección de objetos del paquete modelo mediante ArrayList estáticos, simulando el comportamiento de una base de datos durante la sesión de ejecución. Todas las clases de este paquete son estáticas (sus métodos y atributos son static), lo que permite acceder a la información desde cualquier parte del sistema sin necesidad de instanciar objetos.



Pilar POO evidenciado: Encapsulamiento (colecciones privadas con acceso controlado por métodos)

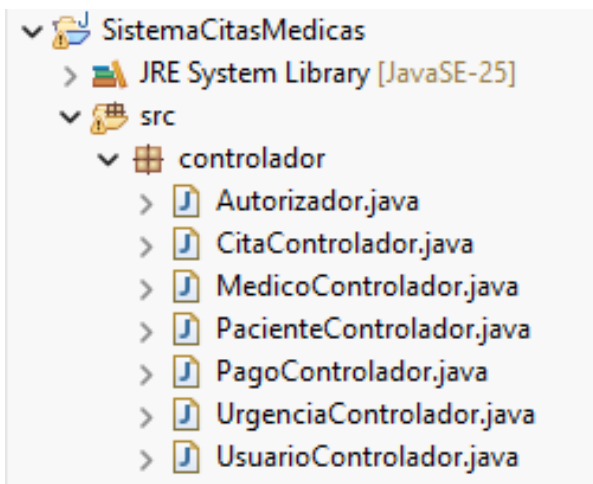
Clase	Descripción
ListaPacientes.java	Almacena todos los objetos Paciente en un ArrayList estático. Provee métodos para agregar, buscar por DNI o nombre, actualizar y eliminar pacientes. Incluye un contador autoincremental de IDs.
ListaMedicos.java	Gestiona la colección de objetos Medico. Permite buscar médicos disponibles, filtrar por especialidad y verificar duplicidad de CMP antes de registrar.
ListaEspecialidades.java	Contiene las especialidades médicas precargadas (Medicina General, Pediatría, Cardiología, etc.) y permite agregar o eliminar especialidades en tiempo de ejecución.
ListaHorarios.java	Almacena los horarios de los médicos. Permite verificar si un médico tiene disponibilidad en un día y turno específicos, funcionalidad requerida al programar una cita.
ListaCitas.java	Administra todas las citas del sistema. Implementa la validación de cruce de horarios: antes de agregar una cita, verifica que no exista otra confirmada para el mismo médico en la misma fecha y hora.
ListaUrgencias.java	Gestiona los registros de urgencias activas. Permite filtrar urgencias por estado (En atención / Alta médica).
ListaPagos.java	Almacena los pagos registrados. Provee métodos para consultar pagos filtrados por fecha y calcular totales de ingresos por periodo.
ListaUsuarios.java	Contiene los usuarios precargados del sistema (Administrador, Recepcionista, Médico, Cajero, Paciente). El método autenticar() recorre la lista y valida las credenciales ingresadas.

A continuación, se muestra el fragmento de código de ListaCitas.java que implementa la validación de cruce de horarios, uno de los requerimientos funcionales más importantes del sistema (RF-20):

```
public static boolean existeConflicto(int medicoId, String fecha, String hora, int citaIdExcluir) {
    String sql = "SELECT COUNT(*) FROM citas WHERE medico_id = ? AND fecha = ? AND hora = ?"
        + " AND estado != 'Cancelada' AND id != ?";
    try (Connection con = DatabaseConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, medicoId);
        ps.setString(2, fecha);
        ps.setString(3, hora);
        ps.setInt(4, citaIdExcluir);
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() && rs.getInt(1) > 0;
        }
    } catch (SQLException ex) {
        throw new RuntimeException("Error al verificar conflicto de horario", ex);
    }
}
```

13.3.3 Paquete controlador

El paquete controlador implementa la lógica de negocio del sistema. Las clases de este paquete actúan como intermediarias entre la capa de vista (interfaz gráfica) y la capa de datos (colecciones en memoria). Cada controlador aplica validaciones antes de delegar la operación a la clase de datos correspondiente, garantizando la integridad de la información.



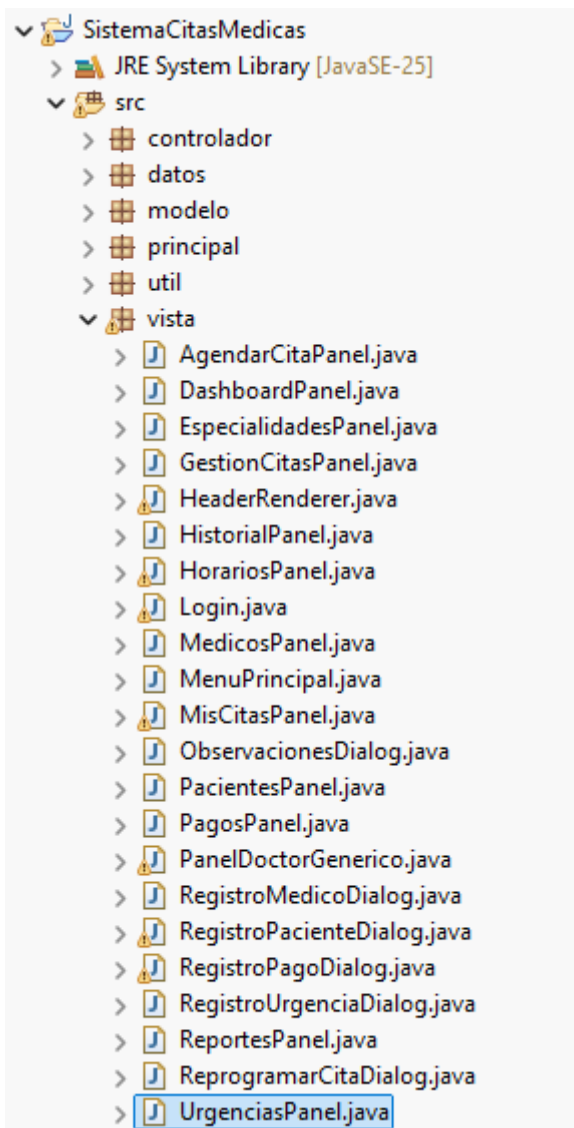
Clase	Descripción
Autorizador.java	Define una matriz de permisos centralizada. El método estático puedeAcceder(rol, modulo) determina si un rol dado tiene acceso a un módulo específico del sistema. Es

Clase	Descripción
	consultado por todos los paneles de la vista antes de mostrar u ocultar opciones de menú.
UsuarioControlador.java	Gestiona la autenticación de usuarios. Delega la validación de credenciales a ListaUsuarios y retorna el objeto Usuario autenticado junto con su rol para que la vista configure el menú correspondiente.
PacienteControlador.java	Orquesta las operaciones sobre pacientes: registro, edición, eliminación y búsqueda. Valida que el DNI tenga exactamente 8 dígitos y que no existan registros duplicados antes de persistir.
MedicoControlador.java	Gestiona médicos y horarios. Valida el formato del CMP, la unicidad del DNI y la asignación de especialidad. Provee los métodos necesarios para verificar la disponibilidad de un médico en el flujo de agendamiento de citas.
CitaControlador.java	Controla el ciclo de vida completo de las citas: programación, cancelación, reprogramación y actualización de estado. Invoca la validación de cruce de horarios en ListaCitas antes de confirmar cada programación. También provee los métodos de consulta para los módulos de historial y reportes.
UrgenciaControlador.java	Gestiona el registro de urgencias, la asignación de médico disponible y el alta médica de cada caso de urgencia activo.
PagoControlador.java	Registra pagos asociados a citas, valida que una cita no sea pagada más de una vez y actualiza automáticamente su estado a Pagado. Provee cálculos de ingresos por periodo para el módulo de reportes.

13.3.4 Paquete vista

El paquete vista contiene todas las clases de interfaz gráfica del sistema, implementadas con la biblioteca Java Swing. Este paquete es el más extenso del proyecto, que cubren los formularios, paneles y diálogos de cada módulo del sistema. Las clases de vista no contienen lógica de negocio: únicamente capturan datos del usuario, los pasan al controlador correspondiente y muestran los resultados.

Integra todos los formularios, paneles y diálogos que conforma la interfaz gráfica del usuario, desarrollada con Java Swing.



Pilar POO evidenciado: Encapsulamiento (la vista solo conoce a los controladores, no accede directamente a los datos)

Clase	Descripción
Login.java	Formulario de inicio de sesión. Captura usuario y contraseña, invoca a UsuarioControlador para autenticar y redirige al MenuPrincipal según el rol obtenido.
MenuPrincipal.java	Ventana principal del sistema. Consulta a Autorizador.puedeAcceder() para mostrar u ocultar los botones de cada módulo según el rol del usuario autenticado.
DashboardPanel.java	Panel de resumen. Muestra tarjetas con el total de citas del día, ingresos del día, citas pendientes de pago y urgencias activas.
PacientesPanel.java	Panel de gestión de pacientes con tabla de búsqueda en tiempo real y botones de edición y eliminación.
RegistroPacienteDialog.java	Diálogo de registro y edición de pacientes. Calcula y muestra la edad automáticamente al ingresar la fecha de nacimiento.
MedicosPanel.java	Panel de gestión de médicos con tabla y opciones de registro, edición y eliminación.
RegistroMedicoDialog.java	Diálogo de registro y edición de médicos. Incluye un selector de especialidad que se carga dinámicamente desde ListaEspecialidades.
EspecialidadesPanel.java	Panel de gestión de especialidades médicas con registro y eliminación.
HorariosPanel.java	Panel de horarios con filtro por médico. Muestra en tiempo real los horarios disponibles. Solo el Administrador puede agregar o eliminar horarios.
AgendarCitaPanel.java	Formulario de programación de citas. Invoca la validación de cruces de horario antes de confirmar.
GestionCitasPanel.java	Panel principal de citas con filtros por fecha, médico y paciente. Permite actualizar el estado de cada cita.
ReprogramarCitaDialog.java	Diálogo para capturar la nueva fecha y hora de una cita reprogramada.
UrgenciasPanel.java	Panel de urgencias activas. Muestra los registros en color rojo para diferenciación visual.
RegistroUrgenciaDialog.java	Diálogo de registro de urgencias con selector de médico disponible.

Clase	Descripción
HistorialPanel.java	Panel de consulta del historial clínico de un paciente, con todas sus atenciones y observaciones registradas.
ObservacionesDialog.java	Diálogo para que el médico registre el motivo y observaciones de una cita atendida.
PagosPanel.java	Panel del módulo de caja. Muestra citas pendientes de pago en naranja y el historial de pagos registrados con filtro por fecha.
RegistroPagoDialog.java	Diálogo de registro de pago. Captura el monto y el método de pago, actualiza el estado de la cita a Pagado.
ReportesPanel.java	Panel de reportes con tres secciones: reporte diario de citas, reporte por médico y reporte de ingresos por método de pago.
MisCitasPanel.java	Panel exclusivo para el rol Paciente. Muestra únicamente las citas del paciente autenticado.
PanelDoctorGenerico.java	Panel base para la vista del médico. Muestra las citas asignadas y permite registrar observaciones.
HeaderRenderer.java	Clase auxiliar que personaliza el encabezado visual de las tablas JTable en toda la interfaz gráfica.

13.3.5 Paquete Util

El paquete util fue incorporado en la versión actual del sistema para dar soporte a la conexión con la base de datos MySQL mediante JDBC (Java Database Connectivity). Este paquete contiene la clase DatabaseConnection, que centraliza la lógica de configuración y gestión de la conexión, siguiendo el patrón Singleton para garantizar que en todo momento exista una única conexión activa a la base de datos.

Clase	Descripción
DatabaseConnection.java	Gestiona la conexión JDBC a MySQL. Lee los parámetros de conexión desde el archivo config.properties y provee los métodos estáticos getConnection() y closeConnection() para que los controladores puedan interactuar con la base de datos sin conocer los detalles de configuración.

La clase DatabaseConnection implementa tres responsabilidades bien definidas que evidencian el principio de encapsulamiento:

1. Carga de configuración: El bloque static { cargarConfiguracion(); } se ejecuta una sola vez cuando la clase es cargada por la JVM. Lee el archivo config.properties desde varias rutas posibles y construye la cadena de conexión JDBC. Si no encuentra el archivo, usa valores por defecto.

2. Conexión única (patrón Singleton): El atributo connection es privado y estático. El método getConnection() verifica si la conexión es nula o está cerrada antes de crear una nueva, garantizando que no se abran conexiones redundantes a la base de datos.

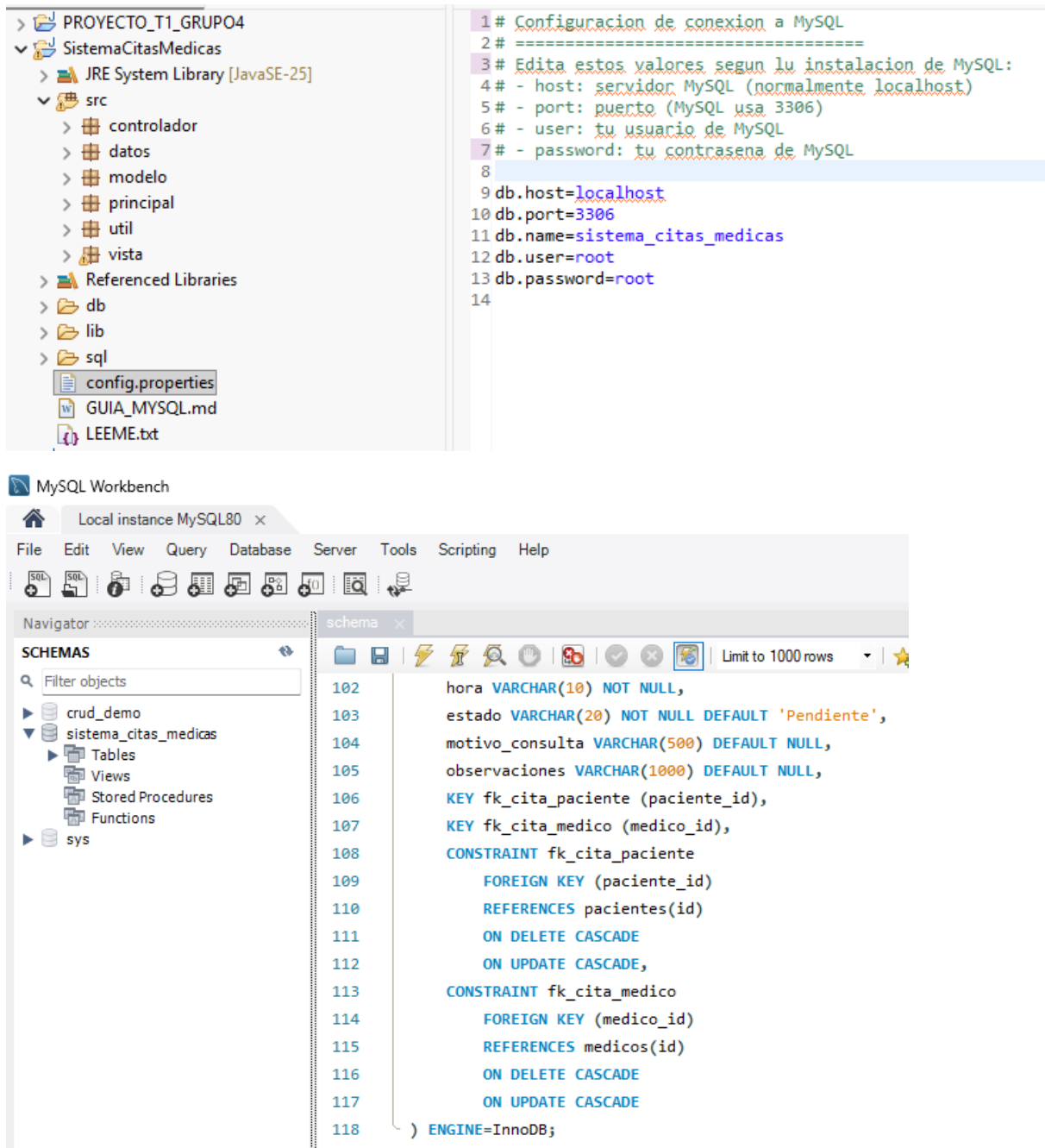
3. Cierre controlado: closeConnection() libera los recursos de la conexión de forma segura, manejando la excepción SQLException sin que el código cliente tenga que preocuparse por ello.

```

9
10 public class DatabaseConnection {
11
12     private static String URL;
13     private static String USER;
14     private static String PASSWORD;
15
16     private static Connection connection = null;
17
18     static {
19         cargarConfiguracion();
20     }
21
22     private static void cargarConfiguracion() {
23         Properties props = new Properties();
24         String ruta = "config.properties";
25
26         // Busca el archivo en varias ubicaciones posibles
27         String[] rutas = {
28             ruta,
29             "src/" + ruta,
30             "../" + ruta,
31             System.getProperty("user.dir") + "/" + ruta,
32             System.getProperty("user.dir") + "/src/" + ruta
33         };
34
35         boolean cargado = false;
36         for (String path : rutas) {
37             try (InputStream input = new FileInputStream(path)) {
38                 props.load(input);
39                 cargado = true;
40                 break;
41             } catch (Exception ignored) {
42             }
43         }
44
45         if (!cargado) {
46             // Valores por defecto si no encuentra el archivo
47             URL = "jdbc:mysql://localhost:3306/sistema_citas_medicas?useSSL=false&serverTimezone=UTC";
48             USER = "root";
49             PASSWORD = "root";
50             return;
51         }
52
53         String host = props.getProperty("db.host", "localhost");
54         String port = props.getProperty("db.port", "3306");
55         String dbName = props.getProperty("db.name", "sistema_citas_medicas");
56         USER = props.getProperty("db.user", "root");
57         PASSWORD = props.getProperty("db.password", "root");
58
59         URL = "jdbc:mysql://" + host + ":" + port + "/" + dbName
60             + "?useSSL=false&serverTimezone=UTC";
61     }
62
63     public static Connection getConnection() {
64         try {
65             if (connection == null || connection.isClosed()) {
66                 Class.forName("com.mysql.cj.jdbc.Driver");
67                 connection = DriverManager.getConnection(URL, USER, PASSWORD);
68             }
69         } catch (ClassNotFoundException e) {
70             throw new RuntimeException("No se encontró el driver JDBC de MySQL. " +
71                 "Agrega mysql-connector-java.jar al classpath.", e);
72         } catch (SQLException e) {
73             throw new RuntimeException("Error al conectar a la base de datos MySQL.\n" +
74                 "Verifica que MySQL esté ejecutándose en localhost:3306\n" +
75                 "y que la base de datos 'sistema_citas_medicas' exista.", e);
76         }
77         return connection;
78     }
79
80     public static void closeConnection() {
81         if (connection != null) {
82             try {
83                 connection.close();
84                 connection = null;
85             } catch (SQLException e) {
86                 System.err.println("Error al cerrar conexión: " + e.getMessage());
87             }
88         }
89     }
90 }
91

```

El archivo config.properties, ubicado en la raíz del proyecto, contiene los parámetros configurables sin necesidad de modificar el código fuente:



13.3.6 Paquete Principal

La clase Main, ubicada en el paquete principal, es el punto de entrada del sistema. Su única responsabilidad es configurar el aspecto visual de la interfaz (Look and Feel del sistema operativo) e iniciar la ventana de login en el hilo de eventos de Swing mediante `SwingUtilities.invokeLater()`. Esta separación garantiza que la interfaz gráfica se inicialice en el hilo correcto, evitando problemas de concurrencia propios de las aplicaciones Swing.

```

1 package principal;
2
3 import javax.swing.SwingUtilities;
4
5 public class Main {
6
7     public static void main(String[] args) {
8         try {
9             UIManager.setLookAndFeel(
10                 UIManager.getSystemLookAndFeelClassName()
11             );
12         } catch (Exception e) {
13             e.printStackTrace();
14         }
15
16         // Verificar conexión a MySQL antes de iniciar
17         try {
18             DatabaseConnection.getConnection();
19         } catch (Exception e) {
20             JOptionPane.showMessageDialog(null,
21                 "Error al conectar con MySQL.\n\n" +
22                 "1. Asegúrate de que MySQL esté ejecutándose.\n" +
23                 "2. Ejecuta el script sql/schema.sql en MySQL Workbench.\n" +
24                 "3. Verifica usuario/password en DatabaseConnection.java.\n\n" +
25                 "Detalle: " + e.getMessage(),
26                 "Error de conexión",
27                 JOptionPane.ERROR_MESSAGE);
28             System.exit(1);
29         }
30
31         SwingUtilities.invokeLater(new Runnable() {
32             @Override
33             public void run() {
34                 Login login = new Login();
35                 login.setVisible(true);
36             }
37         });
38     }
39 }

```

13.3.7 Interfaz general del proyecto modo Administrador

Clínica Grupo_04 — Sistema de Citas Médicas

Bienvenido

Carlos Admin
Rol: Administrador

REGISTROS

- Pacientes
- Médicos
- Especialidades
- Horarios

ATENCION

- Agendar Cita
- Caja — Pagos
- Consultorio
- Urgencias

CLINICA

- Historial
- Reportes

INICIO

- Dashboard
- Cerrar sesión

Dashboard

Resumen del sistema

2 Pacientes

3 Médicos

0 Citas

0 Pend. Pago

0 Urgencias

0 Ingresos

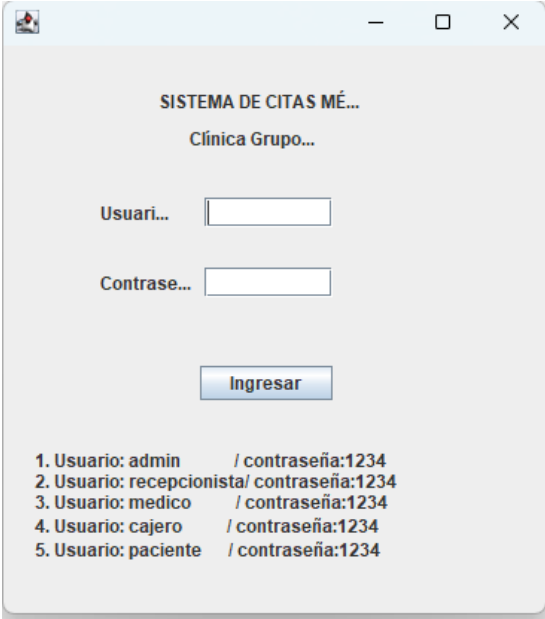
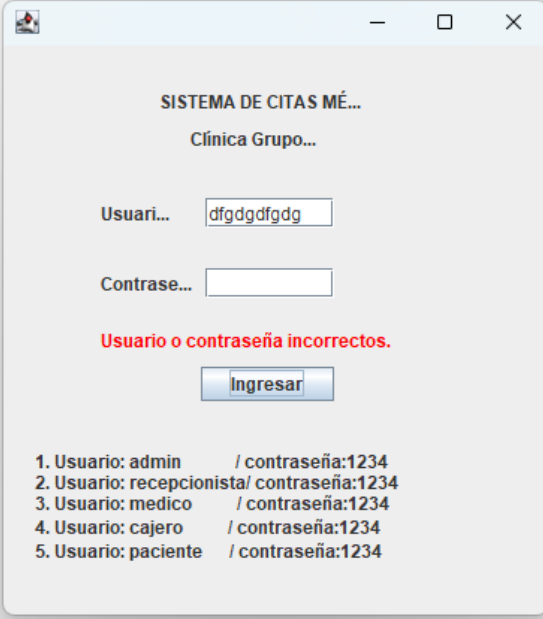
Citas registradas

Paciente	Doctor	Hora	Motivo	Estado

Urgencias activas

Paciente	Motivo	Prioridad	Doctor	Estado

13.4 Capturas de ejecución del sistema y validación de historias de usuario implementados:

ID	Título	Descripción	Criterios de Aceptación	Estado
US-01	Iniciar sesión	Como usuario del sistema (administrador, recepcionista, médico o cajero), quiero ingresar mis credenciales de usuario y contraseña, para acceder únicamente a los módulos correspondientes a mi rol sin que otros usuarios puedan ver mi información.	Credenciales correctas dan acceso al panel del rol asignado. Credenciales incorrectas muestran mensaje de error sin conceder acceso. Requerimientos cubiertos: RF-01, RF-02	☑
 				
US-02	Recuperar contraseña olvidada	Como usuario registrado en el sistema, quiero recuperar mi contraseña ingresando mi DNI y correo electrónico, para poder volver a acceder al sistema sin necesidad de contactar al administrador	Datos válidos permiten establecer una nueva contraseña. Datos incorrectos muestran mensaje de error sin modificar la contraseña. Requerimientos cubiertos: RF-03	✗

US-03	Registrar un nuevo paciente	Como recepcionista, quiero registrar los datos personales de un nuevo paciente (nombre, DNI, fecha de nacimiento, sexo, teléfono, correo y tipo de seguro), para tener su información disponible en el sistema y poder programarle citas médicas.	El registro se guarda correctamente y aparece ventana con mensaje "Paciente registrado correctamente" mostrando todos los datos ingresados. DNI duplicado o campos obligatorios vacíos muestran mensaje de error específico. Requerimientos cubiertos: RF-04	☑
-------	-----------------------------	---	--	---

Registro de Pacientes
Gestión de pacientes — RF-04 a RF-08

Buscar: + Nuevo...

ID	Nombre	DNI	Edad	Sexo	Teléfono	Seguro
1	Carlos Rodríguez	70123456	35	Masculino	987654321	Particular
2	Ana Martínez	70123457	40	Femenino	987111222	EsSalud

Registro de Paciente

REGISTRO DE PACIENTE

Nombre: *

Apellido: *

DNI: *

F. Nacimiento: * 28 años

Sexo: * ☒ Masculino ☐ Femenino

Teléfono: *

Correo: *

Tipo Seguro: *

* Campos obligatorios / La edad se calcula automáticamente

Registrar Paciente Cancelar

Registro exitoso

i Paciente Pedro Martinez registrado correctamente.

Aceptar

US-04	Editar datos de un paciente existente	Como recepcionista, quiero modificar los datos de un paciente ya registrado, para mantener la información actualizada cuando el paciente cambia su teléfono, dirección o tipo de seguro.	Los cambios se reflejan de forma inmediata en el sistema. No se permite guardar si hay campos obligatorios vacíos. Requerimientos cubiertos: RF-05	☑
-------	---------------------------------------	--	--	---

Bienvenido
Carlos Admin
Rol: Administrador

REGISTROS

- Pacientes
- Médicos
- Especialidades
- Horarios

ATENCION

- Agendar Cita
- Caja — Pagos
- Consultorio
- Urgencias

CLINICA

- Historial
- Reportes

INICIO

- Dashboard
- Cerrar sesión

Registro de Pacientes
Gestión de pacientes — RF-04 a RF-08

Buscar:

[+ Nuevo...](#)

ID	Nombre	DNI	Edad	Sexo	Teléfono	Seguro
1	Carlos Rodríguez	70123456	35	Masculino	987654321	Particular
2	Ana Martínez	70123457	40	Femenino	987111222	EsSalud
3	Pedro Martínez	65895847	28	Masculino	946591254	SIS

Editar Paciente

EDITAR PACIENTE

Nombre: *

Apellido: *

DNI: *

F. Nacimiento: * 28 años

Sexo: * ☒ Masculino ☐ Femenino

Teléfono: *

Correo: *

Tipo Seguro: *

* Campos obligatorios / La edad se calcula automáticamente

[Guardar cambios](#) [Cancelar](#)

Actualizado

Datos del paciente actualizados correctamente.

[Aceptar](#)

US-05	Buscar paciente por DNI	Como recepcionista, quiero buscar a un paciente escribiendo su número de DNI, para localizarlo rápidamente al momento de registrar una cita o verificar sus datos sin tener que revisar toda la lista.	La búsqueda retorna el registro del paciente si existe. Si el DNI no está registrado, se muestra mensaje "Paciente no encontrado". Requerimientos cubiertos: RF-07, RF-08	☑
-------	-------------------------	--	---	---

Bienvenido
Ana Recepción
Rol: Recepcionista

REGISTROS

- Pacientes
- Horarios

ATENCION

Registro de Pacientes
Gestión de pacientes — RF-04 a RF-08

Buscar:

[+ Nuevo...](#)

ID	Nombre	DNI	Edad	Sexo	Teléfono	Seguro
3	Pedro Martínez C...	65895847	28	Masculino	946591254	SIS

US-06	Registrar un nuevo médico	Como administrador, quiero registrar los datos de un médico (nombre, DNI, CMP, especialidad, turno y teléfono), para que pueda ser asignado a citas y urgencias dentro del sistema.	El registro se guarda correctamente y aparece ventana con mensaje "Doctor registrado correctamente" mostrando todos los datos. CMP duplicado o campos vacíos muestran mensaje de error. Requerimientos cubiertos: RF-09	☑
-------	---------------------------	---	---	---

Clínica Grupo_04 — Sistema de Citas Médicas

Bienvenido
Ana Recepción
Rol: Recepcionista

Registro de Pacientes
Gestión de pacientes — RF-04 a RF-08

Buscar: 65895847

+ Nuevo...

ID	Nombre	DNI	Edad	Sexo	Teléfono
3	Pedro Martinez C...	55895847	28	Masculino	946591254

US-07	Gestionar especialidades médicas	Como administrador, quiero registrar y listar las especialidades médicas disponibles en la clínica, para poder asignarlas correctamente a cada médico y facilitar la búsqueda de profesionales por área de atención.	Se puede registrar una nueva especialidad sin duplicados. La especialidad queda disponible en el selector al registrar o editar un médico. Requerimientos cubiertos: RF-12, RF-13, RF-14	☒
-------	----------------------------------	--	--	-------------------------------------

Clínica Grupo_04 — Sistema de Citas Médicas

Bienvenido
Carlos Admin
Rol: Administrador

Registro de Médicos
Gestión de médicos — RF-09 a RF-14

+ Nuevo...

ID	Nombre	CMP	Especialidad	Turno	Disponible
1	Juan Pérez	CMP-1001	—	Mañana	Sí
2	Maria García	CMP-1002	—	Tarde	Sí
3	Jorge Smith	5895	Medicina G...	Noche (7p...	Sí

Registro de Médico

Nombre: * Jhon
Apellido: * Altamirano
DNI: * 96584875
CMP: * 5895
Especialidad: * Medicina General
Turno: * Mañana (7am-1pm)
Teléfono: * 965845895
Correo: * doctor2@gmail.com

Ya existe un médico registrado con el CMP 5895.

Aceptar

Registrar Médico Cancelar

US-08	Registrar y gestionar horarios médicos	Como administrador, quiero registrar los horarios de atención de cada médico indicando el día y turno, para que el sistema pueda mostrar la	El horario se asocia al médico y queda visible al agendar citas. No se permite registrar horarios duplicados para el mismo médico, día y turno.	☒
-------	--	---	---	-------------------------------------

disponibilidad real al momento de agendar citas.

No se puede eliminar un horario con citas asignadas.

Requerimientos cubiertos: RF-15, RF-16, RF-17, RF-18

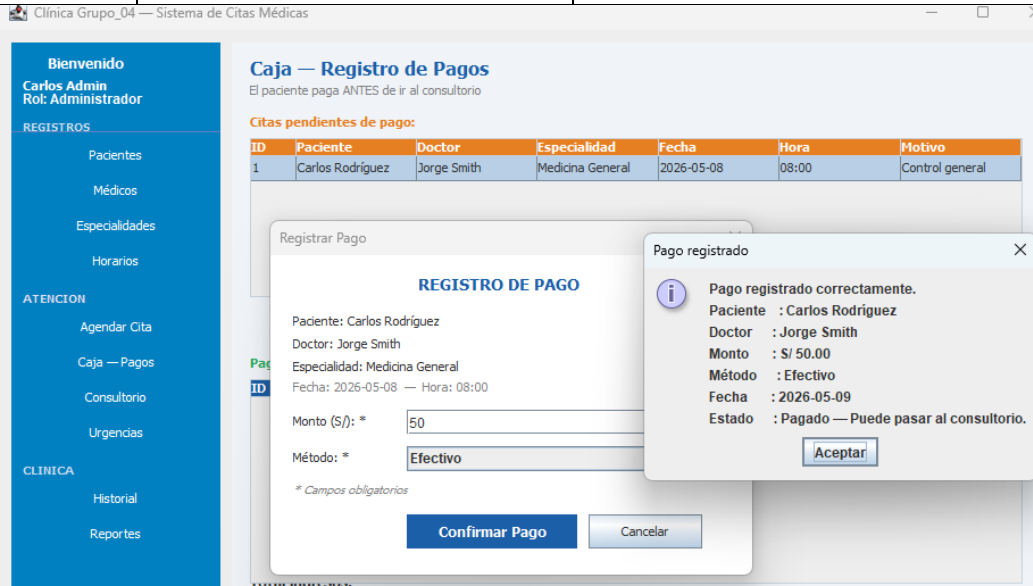
US-09

Programar una cita médica

Como recepcionista, quiero programar una cita seleccionando el paciente, el médico, la fecha y la hora, para organizar la agenda de atenciones de la clínica de forma ordenada y sin conflictos de horario.

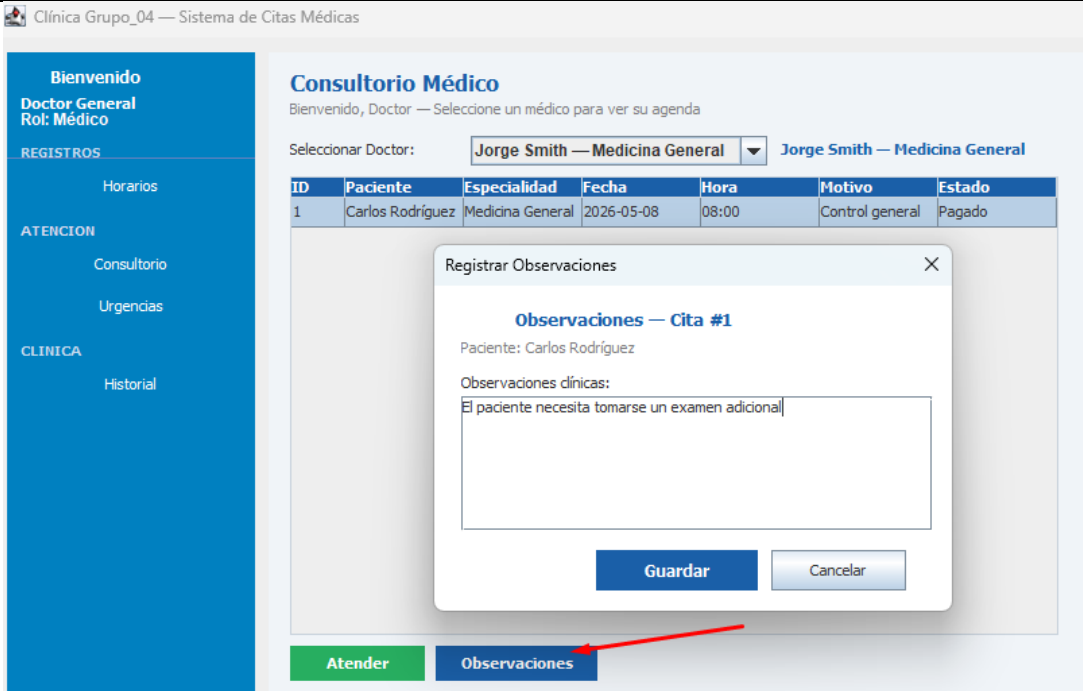
La cita se registra con estado "Pendiente" y aparece ventana de confirmación con todos los datos. Si hay cruce de horario, el sistema muestra alerta y no permite confirmar la cita. Requerimientos cubiertos: RF-19, RF-20



US-10	Cancelar una cita médica	Como recepcionista, quiero cancelar una cita cuando el paciente lo solicita, para liberar el horario del médico y permitir que otro paciente ocupe ese espacio.	El estado de la cita cambia a "Cancelada" y el horario queda disponible. Se registra la fecha y el motivo de cancelación. Si la cita ya estaba cancelada, se muestra mensaje informativo. Requerimientos cubiertos: RF-21, RF-27	☒
				
HU-11	Reprogramar una cita existente	Como recepcionista, quiero reprogramar una cita a una nueva fecha u hora cuando el paciente o el médico no puede asistir, para no perder la atención y mantener la organización de la agenda.	Se valida disponibilidad del nuevo horario antes de confirmar la reprogramación. La cita original queda registrada con estado "Reprogramada" y se crea el nuevo registro. Requerimientos cubiertos: RF-22, RF-26	☒

				
HU-12	Consultar la agenda de citas por fecha y médico	Como recepcionista, quiero consultar todas las citas de un día o de un médico en particular, para tener una visión completa de la agenda y orientar correctamente a los pacientes.	Se puede filtrar por fecha, por médico o por ambos. La lista muestra paciente, médico, hora, motivo y estado de cada cita. Requerimientos cubiertos: RF-23, RF-24, RF-25	☑
				
HU-13	Consultar mis citas del día como médico	Como médico, quiero ver la lista de citas que tengo programadas para el día actual ordenadas por hora, para planificar mi jornada de trabajo con anticipación y atender a los pacientes puntualmente.	Se muestra la agenda del médico con nombre del paciente, hora, motivo y estado. Si no hay citas para el día, se muestra mensaje informativo. Requerimientos cubiertos: RF-24, RF-26	☑

				
HU-14	Atender a un paciente de urgencias	Como médico o recepcionista, quiero registrar a un paciente que llega directamente a urgencias sin cita previa, asignando su prioridad (Alta, Media, Baja) y un médico disponible, para garantizar que reciba atención inmediata sin necesidad de pasar por el proceso regular de citas.	Se registra nombre, DNI, motivo, prioridad y médico asignado. Aparece ventana de confirmación con los datos del paciente enviado a urgencias. Solo se muestran médicos con turno activo disponibles para asignar. Requerimientos cubiertos: RF-28, RF-29	☑
				
HU-15	Registrar observaciones de una consulta	Como médico, quiero registrar el motivo de consulta y mis observaciones clínicas al finalizar cada atención, para	Las observaciones se guardan en el historial vinculadas a la cita, el paciente y el médico.	☑

		dejar constancia en el historial del paciente y que futuras consultas tengan contexto previo.	El campo no puede quedar vacío; se muestra validación si se intenta guardar sin completar. Requerimientos cubiertos: RF-30, RF-31	
				
HU-16	Consultar el historial de atención de un paciente	Como médico, quiero consultar el historial de consultas previas de un paciente antes de atenderlo, para conocer sus antecedentes, diagnósticos anteriores y observaciones de otros médicos que puedan ser relevantes para la nueva consulta.	El historial muestra fecha, médico, motivo, observaciones y estado de pago de cada consulta. La lista está ordenada de más reciente a más antigua. Requerimientos cubiertos: RF-32	☑

 Clínica Grupo_04 — Sistema de Citas Médicas Bienvenido Doctor General Rol: Médico REGISTROS Horarios ATENCION Consultorio Urgencias CLINICA Historial Historial Clínico <table><thead><tr><th>ID</th><th>Paciente</th><th>Doctor</th><th>Fecha</th><th>Motivo</th><th>Observaci...</th><th>Estado</th></tr></thead><tbody><tr><td>1</td><td>Carlos Ro...</td><td>Jorge Smith</td><td>2026-05-08</td><td>Control ge...</td><td>El pacient...</td><td>Atendida</td></tr></tbody></table>					ID	Paciente	Doctor	Fecha	Motivo	Observaci...	Estado	1	Carlos Ro...	Jorge Smith	2026-05-08	Control ge...	El pacient...	Atendida					
ID	Paciente	Doctor	Fecha	Motivo	Observaci...	Estado																	
1	Carlos Ro...	Jorge Smith	2026-05-08	Control ge...	El pacient...	Atendida																	
HU-17	Registrar el pago de una atención	Como cajero, quiero registrar el pago de una cita atendida especificando el monto, el método de pago (efectivo, tarjeta o seguro) y la fecha, para llevar un control preciso de los ingresos de la clínica y que la cita quede marcada como pagada	El pago se registra correctamente y la cita queda con estado "Pagada". Si la cita no existe o ya fue pagada, se muestra mensaje de error. Requerimientos cubiertos: RF-33, RF-34	☒																			
Clínica Grupo_04 — Sistema de Citas Médicas Bienvenido Luis Caja Rol: Cajero ATENCION Caja — Pagos CLINICA Reportes Caja — Registro de Pagos El paciente paga ANTES de ir al consultorio Citas pendientes de pago: <table><thead><tr><th>ID</th><th>Paciente</th><th>Doctor</th><th>Especialidad</th><th>Fecha</th><th>Hora</th><th>Motivo</th></tr></thead><tbody></tbody></table> Registrar Pago Pagos registrados: <table><thead><tr><th>ID</th><th>Paciente</th><th>Doctor</th><th>Fecha Pago</th><th>Monto</th><th>Método</th></tr></thead><tbody><tr><td>1</td><td>Carlos Rodríguez</td><td>Jorge Smith</td><td>2026-05-09</td><td>S/ 50.00</td><td>Efectivo</td></tr></tbody></table> Total ingresos: S/ 50.00					ID	Paciente	Doctor	Especialidad	Fecha	Hora	Motivo	ID	Paciente	Doctor	Fecha Pago	Monto	Método	1	Carlos Rodríguez	Jorge Smith	2026-05-09	S/ 50.00	Efectivo
ID	Paciente	Doctor	Especialidad	Fecha	Hora	Motivo																	
ID	Paciente	Doctor	Fecha Pago	Monto	Método																		
1	Carlos Rodríguez	Jorge Smith	2026-05-09	S/ 50.00	Efectivo																		
HU-18	Eliminar un registro del sistema	Como administrador, quiero eliminar pacientes o médicos que ya no están activos en la	El sistema solicita confirmación antes de eliminar cualquier registro.	☒																			

		clínica, para mantener la base de datos limpia y actualizada	No se permite eliminar un paciente o médico con citas activas asignadas. La eliminación muestra mensaje de confirmación cuando se completa correctamente. Requerimientos cubiertos: RF-06, RF-11	
--	--	--	---	--

Clínica Grupo_04 — Sistema de Citas Médicas

Bienvenido
Carlos Admin
Rol: Administrador

REGISTROS

- Pacientes
- Médicos
- Especialidades
- Horarios

ATENCION

- Agendar Cita
- Caja — Pagos
- Consultorio
- Urgencias

CLINICA

- Historial
- Reportes

INICIO

- Dashboard
- Cerrar sesión

Registro de Pacientes
Gestión de pacientes — RF-04 a RF-08

Buscar: + Nuevo...

ID	Nombre	DNI	Edad	Sexo	Teléfono	Seguro
1	Carlos Rodríguez	70123456	35	Masculino	987654321	Particular
2	Ana Martínez	70123457	40	Femenino	987111222	EsSalud
3	Pedro Martinez Ch...	65895847	28	Masculino	946591254	SIS

Confirmar eliminación

¿Está seguro de eliminar este paciente?

HU-19	Generar reportes de citas e ingresos	Como administrador, quiero generar reportes de citas por fecha o por médico, y reportes de ingresos por periodo, para supervisar el rendimiento de la clínica y tomar decisiones administrativas basadas en datos reales.	El reporte de citas muestra cantidad atendidas, canceladas y reprogramadas. El reporte de ingresos suma los pagos agrupados por método de pago. Si no hay datos para el periodo seleccionado, se muestra mensaje informativo.	☒
-------	--------------------------------------	---	--	-------------------------------------

			Requerimientos cubiertos: RF-35, RF-36, RF-37	
				
HU-20	Validar datos antes de guardar cualquier registro	Como usuario del sistema, quiero que el sistema me avise cuando intento guardar datos incorrectos o incompletos, para evitar errores en la base de datos y mantener la integridad de la información clínica.	Cada campo con error muestra un mensaje descriptivo que indica exactamente qué debe corregir. No se permite guardar ningún registro si hay campos obligatorios vacíos o con formato inválido. Requerimientos cubiertos: RF-38, RF-39, RF-40	☒

Clínica Grupo_04 — Sistema de Citas Médicas

Bienvenido
Carlos Admin
Rol: Administrador

REGISTROS

- Pacientes
- Médicos
- Especialidades
- Horarios

ATENCION

- Agendar Cita
- Caja — Pagos
- Consultorio
- Urgencias

CLINICA

- Historial
- Reportes

Registro de Pacientes
Gestión de pacientes — RF-04 a RF-08

Buscar: Registro de Paciente

REGISTRO DE PACIENTE

Nombre: *
Apellido: *
DNI: *
F. Nacimiento: *
Sexo: * ☒ Masculino ☐ Femenino
Teléfono: *
Correo: *
Tipo Seguro: * SIS

* Campos obligatorios / La edad se calcula automáticamente

Registrar Paciente **Cancelar**

Campo requerido
El nombre es obligatorio.
Aceptar

Registro de Paciente

REGISTRO DE PACIENTE

Nombre: * Marvin
Apellido: *
DNI: *
F. Nacimiento: *
Sexo: * ☒ Ma

Campo requerido
El apellido es obligatorio.
Aceptar

Registro de Paciente

REGISTRO DE PACIENTE

Nombre: * Marvin
Apellido: * Quispe
DNI: *
F. Nacimiento: *
Sexo: * ☒ Ma
Teléfono: *
Correo: *
Tipo Seguro: * SIS

* Campos obligatorios / La edad se calcula automáticamente

Registrar Paciente **Cancelar**

Campo requerido
El DNI debe tener 8 dígitos.
Aceptar

CAPÍTULO XIV: CONCLUSIONES

El desarrollo del Sistema de Atención de Citas Médicas para la Clínica, representó una experiencia de aprendizaje integral que permitió al equipo trasladar los conceptos teóricos de la programación orientada a objetos a una solución de software funcional, capaz de responder a una problemática real identificada en el ámbito de la gestión clínica.

La arquitectura implementada siguiendo el patrón Modelo-Vista-Controlador demostró ser una decisión acertada para organizar el código del sistema. La separación en paquetes independientes permitió que cada integrante del equipo pudiera trabajar en un componente específico sin afectar el funcionamiento de los demás módulos, facilitando el trabajo colaborativo y reduciendo los errores de integración durante el desarrollo.

La aplicación de los principios de la programación orientada a objetos se evidencia de forma concreta a lo largo del sistema. La jerarquía de herencia encabezada por la clase abstracta Persona, de la cual heredan Paciente y Medico, permitió evitar la duplicación de atributos comunes y establecer un patrón claro que cada subclase debe cumplir. El polimorfismo se materializa en la sobrescritura del método mostrarDatos en cada subclase, permitiendo que el sistema trate a distintos tipos de personas de forma uniforme sin conocer su tipo específico en tiempo de compilación. El encapsulamiento protege la integridad de los datos del sistema al restringir el acceso directo a los atributos y canalizarlo exclusivamente a través de métodos controlados.

El sistema logra resolver los problemas centrales identificados en el análisis inicial. La validación automática de cruces de horario elimina la superposición de citas que ocurría en el proceso manual. El registro centralizado de pacientes, médicos y atenciones reemplaza los registros físicos dispersos que generaban pérdida de información. El flujo de estados de cada cita, que transita de Pendiente a Pagada y finalmente a Atendida, garantiza que ninguna consulta sea atendida sin el respaldo administrativo y económico correspondiente, reflejando el proceso real de la clínica.

La integración de JDBC con MySQL como capa de persistencia constituyó uno de los mayores retos técnicos del proyecto, al requerir la comprensión de conceptos como la conexión a base de datos, el uso de PreparedStatement para consultas parametrizadas y el manejo de excepciones propias del acceso a datos. Sin embargo, la arquitectura en capas adoptada desde el inicio facilitó esta integración, ya que los formularios y controladores no requirieron

modificaciones significativas al reemplazar el almacenamiento en memoria por consultas SQL reales.

El módulo de urgencias representó un aporte que surgió del análisis profundo del contexto clínico. Al reconocer que no todos los pacientes pueden seguir el proceso regular de agendamiento, se diseñó e implementó una vía alternativa de atención inmediata con asignación directa de médico y clasificación por nivel de prioridad, demostrando que el levantamiento de requerimientos fue suficientemente exhaustivo como para anticipar escenarios excepcionales.

Como limitaciones del sistema en su versión actual, se identifica que la recuperación de contraseña está contemplada en el diseño, pero no fue implementada en el plazo del proyecto. Asimismo, el control de acceso diferenciado por rol puede fortalecerse en versiones futuras con una matriz de permisos más granular que restrinja funcionalidades específicas dentro de cada módulo. A pesar de estas limitaciones, el sistema constituye una base sólida y funcional sobre la cual pueden desarrollarse mejoras y nuevas funcionalidades en etapas posteriores del proyecto.

CAPÍTULO XV: REFERENCIAS BIBLIOGRÁFICAS

- Acuña, J., & Rosales, M. (2023). *Sistema informático para el área odontológica del subcentro de la parroquia San Francisco – Ecuador*. Universidad Técnica de Ecuador.
- Deitel, P. J. (2012). *Java: cómo programar* (9.^a ed.). Pearson Educación.
- Deitel, P., & Deitel, H. (2016). *Cómo programar en Java* (10.^a ed.). Pearson Educación.
- Durán García, M. A. (2023). *Aprendiendo Git: ¡Domina y comprende Git de una vez por todas!*. Leanpub.
- Gómez, M. A. (2020). *Clean JavaScript: Aprende a aplicar código limpio, SOLID y Testing*. Software Crafters / Leanpub.
- Joyanes Aguilar, L., & Zahonero Martínez, I. (2011). *Programación en Java 6: Algoritmos, programación orientada a objetos e interfaz gráfica de usuario*. McGraw-Hill.
- Llanos Cardona, J. A., López, F. A., & Mejía-Lobo, M. (2023). Implementación de un sistema de seguridad de la información en empresa del sector salud: Caso de estudio. *SUMMA. Revista disciplinaria en ciencias económicas y sociales*, 5(2), 1–14.
<https://doi.org/10.47666/summa.5.2.7>
- Mogollón Moreno, A. A. (2025). *Propuesta de implementación de un sistema de gestión de citas médicas para la Clínica Virgen La Puerta – Tumbes; 2025* [Tesis de pregrado, Universidad Católica Los Ángeles de Chimbote].
- Moure, B. (2024). *Git y GitHub desde cero* (2.^a ed.). Leanpub.
- Ramírez, J., & Vásquez, L. (2022). *Implementación de un sistema digital de gestión de citas médicas en el Hospital Nacional Arzobispo Loayza para mejorar la eficiencia en la atención de los pacientes*. Universidad Nacional Mayor de San Marcos.
- Sánchez González, L. J. (2019). *Aprende Java con ejercicios*. Leanpub.
- Sánchez, P. (2023). *Sistema de gestión de historias clínicas odontológicas y periodontales para la UCACUE*. Universidad Católica de Cuenca.
- Serbat Ocaña, A. (s. f.). *Programación en Java* [Material educativo de referencia].

Veeraiyah, M., Prabakaran, S., Rakasri, L., Vijetha, P., & Vinotha, E. (2026). Smart Web Based Hospital Management System. *Journal of Advances in Developmental Research (IJAIDR)*, 17(1), 1–9.